

CA - repetition - no_frac - Serial position analysis

```
# do this in calling R script
# library(ggplot2)
# library(fmsb) # for TestModels
# library(lme4)
# library(kableExtra)
# library(MASS) # for dropterm
# library(tidyverse)
# library(dominanceanalysis)
# library(cowplot) # for multiple plots in one figure
# library(formatters) # to wrap strings for plot titles
# library(pals) # for continous color palettes

# load needed functions
# source(paste0(RootDir, "/src/function_library/sp_functions.R"))
# do this in the calling R script

# for testing
# CurPat <- "GM"
# CurTask <- "repetition"
# MinLength <- 4
# MaxLength <- 10
# BestModelIndexL1 <- 1
# BestModelIndexL2 <- 1
# BestModelIndexL3 <- 1
# ReportTitle <- paste(CurPat, " - ", CurTask, " - ", RunLabel, " - Serial position analysis")
# RandomSamples <- 10
# DoSimulations <- FALSE
# RemoveFracPres <- TRUE

CurPat <- params$CurPat
CurTask <- params$CurTask
MinLength <- params$MinLength
MaxLength <- params$MaxLength
BestModelIndexL1 <- params$BestModelIndexL1
BestModelIndexL2 <- params$BestModelIndexL2
BestModelIndexL3 <- params$BestModelIndexL3
ReportTitle <- params$ReportTitle
RandomSamples <- params$RandomSamples
DoSimulations <- params$DoSimulations
RemoveFracPres <- params$RemoveFracPres

# done in calling script
# print(paste0("Using root dir: ", RootDir))
# RunDir <- paste0(paste0(RootDir, "/output/"), RunLabel))
# if(!dir.exists(RunDir)){
#   dir.create(RunDir, showWarnings = TRUE)
#   print(paste0("created run directory: ", RunDir))
# }
```

```

# # create patient directory if it doesn't already exist
# PatientDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat)
# if(!dir.exists(PatientDir)){
#   dir.create(PatientDir, showWarnings = TRUE)
#   print(paste0("created participant directory: ", PatientDir))
# }
# TablesDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/tables/")
# if(!dir.exists(TablesDir)){
#   dir.create(TablesDir, showWarnings = TRUE)
#   print(paste0("created tables directory: ", TablesDir))
# }
# FigDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/fig/")
# if(!dir.exists(FigDir)){
#   dir.create(FigDir, showWarnings = TRUE)
#   print(paste0("created figures directory: ", FigDir))
# }
# ReportsDir <- paste0(RootDir, "/output/", RunLabel, "/", CurPat, "/reports/")
# if(!dir.exists(ReportsDir)){
#   dir.create(ReportsDir, showWarnings = TRUE)
#   print(paste0("created reports directory: ", ReportsDir))
# }
results_report_DF <- data.frame(Description=character(), ParameterValue=double())

ModelDatFilename<-paste0(RootDir, "/data/", CurPat, "/", CurPat, "_", CurTask, "_posdat.csv")
if(!file.exists(ModelDatFilename)){
  print("**ERROR** Input data file not found")
  print(sprintf("\tfile: ", ModelDatFilename))
  print(sprintf("\troot dir: ", RootDir))
}
PosDat<-read.csv(ModelDatFilename)

# may already be done in datafile
# if(RemoveFinalPosition){
#   PosDat <- PosDat[PosDat$pos < PosDat$stimlen,] # remove final position
# }
PosDat <- PosDat[PosDat$stimlen >= MinLength,] # include only lengths with sufficient N
PosDat <- PosDat[PosDat$stimlen <= MaxLength,]
# include only correct and nonword errors (not no response, word or w+nw errors)
PosDat <- PosDat[(PosDat$err_type == "correct") | (PosDat$err_type == "nonword"), ]
PosDat <- PosDat[(PosDat$pos) != (PosDat$stimlen), ]
# make sure final positions have been removed
PosDat$stim_number <- NumberStimuli(PosDat)
PosDat$raw_log_freq <- PosDat$log_freq
PosDat$log_freq <- PosDat$log_freq - mean(PosDat$log_freq) # centre frequency
OrigDataLen <- nrow(PosDat)
print(sprintf ("Original data has %d values", OrigDataLen))

## [1] "Original data has 4221 values"

if(RemoveFracPres){ # eliminate fractional preserved values?
  PosDat <- PosDat[(PosDat$preserved == 0) | (PosDat$preserved == 1),]
  DataLen<- nrow(PosDat)
  print(sprintf("Data without fractional preserved values has %d values", DataLen))
  print(sprintf("%d values eliminated.", OrigDataLen - DataLen))
}

```

```
## [1] "Data without fractional preserved values has 4198 values"
## [1] "23 values eliminated."
```

```
PosDat$CumPres <- CalcCumPres(PosDat)
PosDat$CumErr <- CalcCumErrFromPreserved(PosDat)
```

```
# plot distribution of complex onsets and codas
# across serial position in the stimuli -- do complexities concentrate
# on one part of the serial position curve?
# syll_component = 1 is coda
# syll_component = S is satellite
```

```
# get counts of syllable components in different serial positions
```

```
syll_comp_dist_N <- PosDat %>% mutate(pos_factor = as.factor(pos), syll_component_factor = as.factor(syll_component))
```

```
syll_comp_dist_perc <- syll_comp_dist_N %>% ungroup() %>%
  mutate(across(O:S, ~ . / total))
```

```
kable(syll_comp_dist_N)
```

pos_factor	O	P	V	1	S	total
1	533	33	124	NA	NA	690
2	62	NA	427	92	105	686
3	300	NA	161	207	15	683
4	296	NA	230	67	38	631
5	224	NA	205	69	37	535
6	198	1	129	70	21	419
7	168	NA	98	28	17	311
8	90	NA	48	24	4	166
9	68	NA	2	NA	7	77

```
kable(syll_comp_dist_perc)
```

pos_factor	O	P	V	1	S	total
1	0.7724638	0.0478261	0.1797101	NA	NA	690
2	0.0903790	NA	0.6224490	0.1341108	0.1530612	686
3	0.4392387	NA	0.2357247	0.3030747	0.0219619	683
4	0.4690967	NA	0.3645008	0.1061807	0.0602219	631
5	0.4186916	NA	0.3831776	0.1289720	0.0691589	535
6	0.4725537	0.0023866	0.3078759	0.1670644	0.0501193	419
7	0.5401929	NA	0.3151125	0.0900322	0.0546624	311
8	0.5421687	NA	0.2891566	0.1445783	0.0240964	166
9	0.8831169	NA	0.0259740	NA	0.0909091	77

```
# get percentages only from summary table
```

```
syll_comp_perc <- syll_comp_dist_perc[,2:(ncol(syll_comp_dist_perc)-1)]
```

```
syll_comp_perc$pos <- as.integer(as.character(syll_comp_dist_perc$pos_factor))
```

```
syll_comp_perc <- syll_comp_perc %>% rename("Coda" = "1", "Satellite" = "S")
```

```
# pivot to long format for plotting
```

```
syll_comp_plot_data <- syll_comp_perc %>% pivot_longer(cols=seq(1:(ncol(syll_comp_perc)-1)),
  names_to="syll_component", values_to="percent") %>% filter(syll_component %in% c("Coda", "Satellite"))
```

```
# plot satellite and coda occurrences across position

syll_comp_plot <- ggplot(syll_comp_plot_data,
                        aes(x=pos,y=percent,group=syll_component,
                           color=syll_component,
                           linetype = syll_component,
                           shape = syll_component)) +
  geom_line() + geom_point() + scale_color_manual(name="Syllable component", values = palette_values) +
syll_comp_plot <- syll_comp_plot + scale_y_continuous(name="Percent of segment types") + scale_linetype_manual(name="Syllable component", values = palette_linetypes) +
  scale_shape_discrete(name="Syllable component")
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_pos_of_syll_complexities_in_stimuli.png"), plot=syll_comp_plot)
```

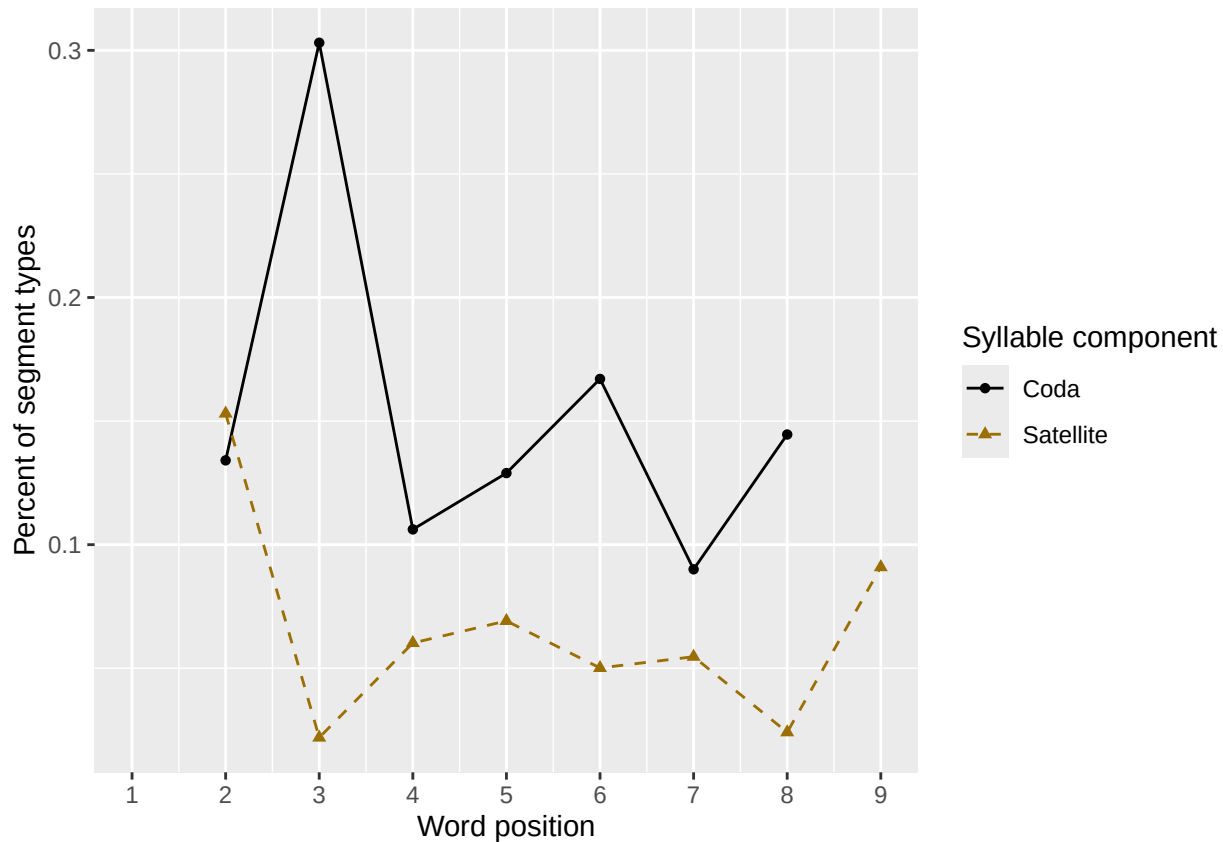
Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_line()``).

Warning: Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).

syll_comp_plot

```
## Warning: Removed 3 rows containing missing values or values outside the scale range (`geom_line()`).
```

Removed 3 rows containing missing values or values outside the scale range (``geom_point()``).



```
# len/pos table
pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(preserved = mean(preserved))

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

min_preserved_raw <- min(as.numeric(unlist(pos_len_summary$preserved)))
max_preserved_raw <- max(as.numeric(unlist(pos_len_summary$preserved)))
preserved_range <- (max_preserved_raw - min_preserved_raw)
```

```

min_preserved <- min_preserved_raw - (0.1 * preserved_range)
max_preserved <- max_preserved_raw + (0.1 * preserved_range)
pos_len_table <- pos_len_summary %>% pivot_wider(names_from = pos, values_from = preserved)
write.csv(pos_len_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask, "_preserved_percentages_by_len",
pos_len_table

```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.983 1      1      NA      NA      NA      NA      NA      NA
## 2      5 0.968 0.968 1      0.989 NA      NA      NA      NA      NA
## 3      6 0.965 0.983 0.974 0.965 0.948 NA      NA      NA      NA
## 4      7 0.963 0.963 0.972 0.982 0.972 0.982 NA      NA      NA
## 5      8 0.932 0.918 0.952 0.946 0.972 0.958 0.952 NA      NA
## 6      9 0.9    0.876 0.899 0.933 0.878 0.933 0.933 0.922 NA
## 7     10 0.896 0.946 0.904 0.895 0.896 0.934 0.883 0.908 0.883

```

```
# len/pos table
```

```
pos_len_N <- PosDat %>% group_by(stimlen, pos) %>% summarise(preserved = mean(preserved), N=n()) %>% dplyr::
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```

pos_len_N_table <- pos_len_N %>% pivot_wider(names_from = pos, values_from = N)
write.csv(pos_len_N_table, paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
"_N_by_len_position.csv"))

```

```
pos_len_N_table
```

```

## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen  `1`  `2`  `3`  `4`  `5`  `6`  `7`  `8`  `9`
##   <int> <int> <int> <int> <int> <int> <int> <int> <int> <int>
## 1      4    58    58    58    NA    NA    NA    NA    NA    NA
## 2      5    94    94    94    94    NA    NA    NA    NA    NA
## 3      6   115   115   115   115   115    NA    NA    NA    NA
## 4      7   109   109   109   109   109   109    NA    NA    NA
## 5      8   147   147   145   147   144   144   145    NA    NA
## 6      9    90    89    89    90    90    90    89    90    NA
## 7     10    77    74    73    76    77    76    77    76    77

```

```

obs_linetypes <- c("solid", "solid", "solid", "solid",
"solid", "solid", "solid", "solid", "solid")

```

```
pred_linetypes <- "dashed"
```

```
pos_len_summary$stimlen <- factor(pos_len_summary$stimlen)
```

```
pos_len_summary$pos <- factor(pos_len_summary$pos)
```

```
# len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
```

```

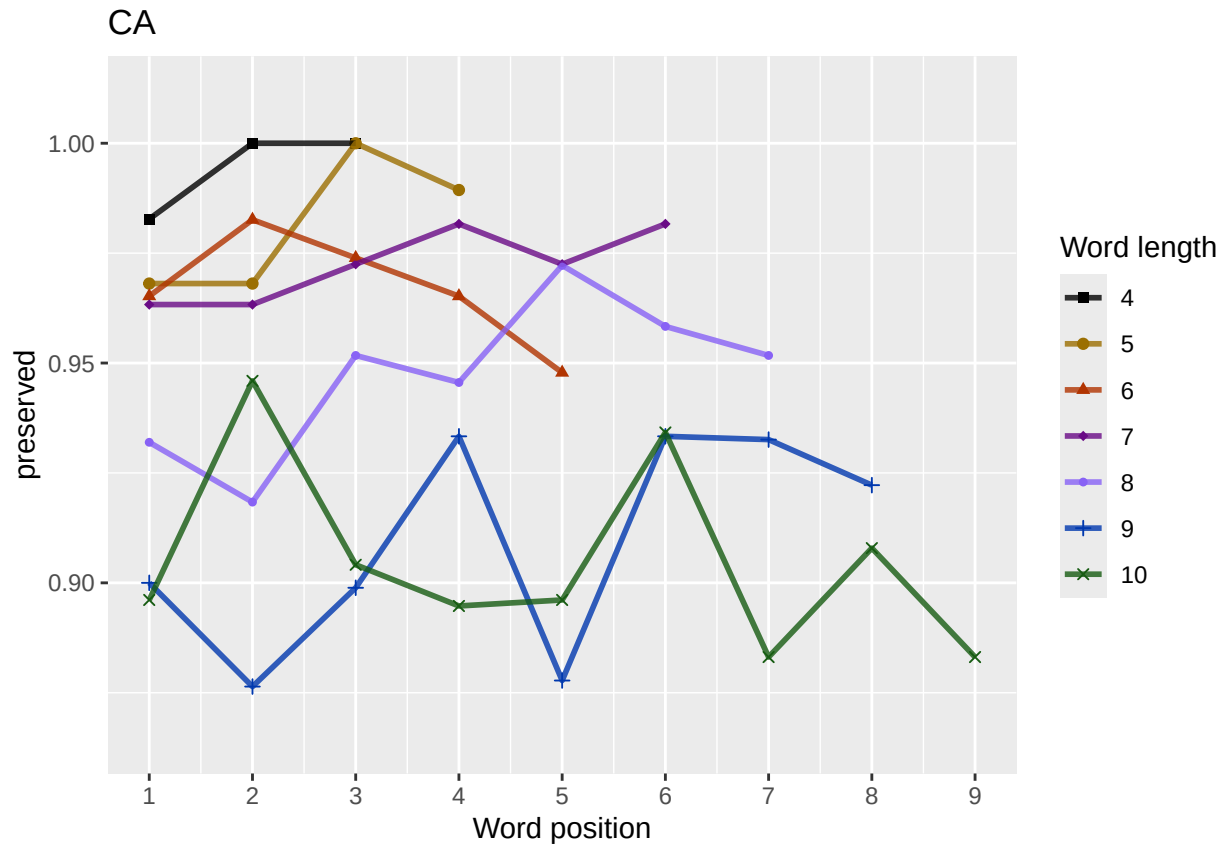
len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
paste0(CurPat),
NULL,
c(min_preserved, max_preserved),
palette_values,
shape_values,
obs_linetypes,
pred_linetypes)

```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
```

```
ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask,
              "_percent_preserved_by_length_pos.png"),
       plot=len_pos_plot, device="png", unit="cm", width=15, height=11)
len_pos_plot
```



Length and position

```
# length and position
```

```
LPMModelEquations<-c("preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)
```

```
LPres<-TestModels(LPMModelEquations,PosDat)
```

```
## *****
```

```
## model index: 2
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```

##
## Coefficients:
## (Intercept)      stimlen
##      5.8316      -0.3693
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -854.1651
## Nagelkerke R2:  0.04729663
## % pres/err predicted correctly:  -425.1153
## % of predictable range [ (model-null)/(1-null) ]:  0.01593689
## *****
## model index:  4
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      5.83090      -0.38827      0.03868
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1707  AIC: 1713
## log likelihood:  -853.4134
## Nagelkerke R2:  0.04831697
## % pres/err predicted correctly:  -424.9824
## % of predictable range [ (model-null)/(1-null) ]:  0.01624368
## *****
## model index:  7
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      5.54224      -0.37944      -0.01515      0.17281
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1706  AIC: 1714
## log likelihood:  -852.761
## Nagelkerke R2:  0.0492022
## % pres/err predicted correctly:  -424.808
## % of predictable range [ (model-null)/(1-null) ]:  0.01664653
## *****
## model index:  5
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos  stimlen:pos

```

```

##      5.1397      -0.3095      0.2568      -0.0243
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1706 AIC: 1714
## log likelihood: -852.921
## Nagelkerke R2:  0.04898515
## % pres/err predicted correctly: -424.811
## % of predictable range [ (model-null)/(1-null) ]:  0.01663952
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)      pos  stimlen:I(pos^2)      stimlen
##      4.725517      -0.281959      -0.062484      0.635549      0.005506      -0.0
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4192 Residual
## Null Deviance:      1777
## Residual Deviance: 1705 AIC: 1717
## log likelihood: -852.5377
## Nagelkerke R2:  0.04950514
## % pres/err predicted correctly: -424.782
## % of predictable range [ (model-null)/(1-null) ]:  0.01670642
## *****
## model index:  6
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)      pos
##      2.6091      -0.0329      0.2332
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1767 AIC: 1773
## log likelihood: -883.7294
## Nagelkerke R2:  0.006874509
## % pres/err predicted correctly: -430.8122
## % of predictable range [ (model-null)/(1-null) ]:  0.002780514
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      pos
##      3.09117      -0.06055
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual

```



```
## Null Deviance:      1777
## Residual Deviance: 1774 AIC: 1778
## log likelihood:    -886.8559
## Nagelkerke R2:    0.002566396
## % pres/err predicted correctly:  -431.5924
## % of predictable range [ (model-null)/(1-null) ]:  0.0009788256
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)
##          2.853
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4197 Residual
## Null Deviance:      1777
## Residual Deviance: 1777 AIC: 1779
## log likelihood:    -888.7162
## Nagelkerke R2:    3.21633e-16
## % pres/err predicted correctly:  -432.0162
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestLPModel<-LPRes$ModelResult[[1]]
BestLPModelFormula<-LPRes$Model[[1]]

LPAICSummary<-data.frame(Model=LPRes$Model,
                          AIC=LPRes$AIC,
                          row.names = LPRes$Model)
LPAICSummary$DeltaAIC<-LPAICSummary$AIC-LPAICSummary$AIC[1]
LPAICSummary$AICexp<-exp(-0.5*LPAICSummary$DeltaAIC)
LPAICSummary$AICwt<-LPAICSummary$AICexp/sum(LPAICSummary$AICexp)
LPAICSummary$NagR2<-LPRes$NagR2

LPAICSummary <- merge(LPAICSummary,LPRes$CoefficientValues, by='row.names',sort=FALSE)
LPAICSummary <- subset(LPAICSummary, select = -c(Row.names))

write.csv(LPAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                              "_len_pos_model_summary.csv"),row.names = FALSE)

kable(LPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2 (Intercept)	stimlen	pos	stimlen:pos	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen	1712.330	0.0000000	0.0000000	0.0345547	0.8047296	6831551	-	NA	NA	NA
						0.3693428				
preserved ~ stimlen + pos	1712.827	0.4966343	0.7801119	0.5269560	0.2048315	0.830899	-	0.0386765	NA	NA
						0.3882686				
preserved ~ stimlen + I(pos^2)	1713.522	1.1919095	0.5510362	0.1904094	0.4920325	0.42242	-	0.1728132	NA	NA
						0.3794447			0.0151525	
+ pos										
preserved ~ stimlen * pos	1713.842	1.5118571	0.4695704	0.1622604	0.4898511	0.39682	-	0.2567670	-	NA
						0.3095467		0.0243008		

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1717.074	1.745323	3809323	0.2032216	0.495031	1725517	-	0.6355486	-	-	0.0055065
							0.2819588		0.0544490	0.0624842	
preserved ~ I(pos^2) + pos	1773.459	1.128641	10000000	0.0000000	0.006872	5609117	NA	0.2331777	NA	-	NA
										0.0328963	
preserved ~ pos	1777.716	5.381630	10000000	0.0000000	0.0025664	091172	NA	-	NA	NA	NA
								0.0605511			
preserved ~ 1	1779.432	7.102189	10000000	0.0000000	0.0000000	00852547	NA	NA	NA	NA	NA

```
print(BestLPModelFormula)
```

```
## [1] "preserved ~ stimlen"
```

```
print(BestLPModel)
```

```
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
```

```
## data = PosDat)
```

```
##
```

```
## Coefficients:
```

```
## (Intercept)      stimlen
```

```
##      5.8316      -0.3693
```

```
##
```

```
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
```

```
## Null Deviance:      1777
```

```
## Residual Deviance: 1708 AIC: 1712
```

```
PosDat$LPFitted<-fitted(BestLPModel)
```

```
fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat, PosDat$patient[1],  
NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
fitted_pos_len_summary <- PosDat %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfitted))
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
fitted_pos_len_table <- fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_from = fitted)
```

```
fitted_pos_len_table
```

```
## # A tibble: 7 x 10
```

```
## # Groups:   stimlen [7]
```

```
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`  
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
```

```
## 1      4 0.987 0.987 0.987 NA      NA      NA      NA      NA      NA
```

```
## 2      5 0.982 0.982 0.982 0.982 NA      NA      NA      NA      NA
```

```
## 3      6 0.974 0.974 0.974 0.974 0.974 NA      NA      NA      NA
```

```
## 4      7 0.963 0.963 0.963 0.963 0.963 0.963 NA      NA      NA
```

```
## 5      8 0.947 0.947 0.947 0.947 0.947 0.947 0.947 NA      NA
```

```
## 6      9 0.925 0.925 0.925 0.925 0.925 0.925 0.925 0.925 NA
```

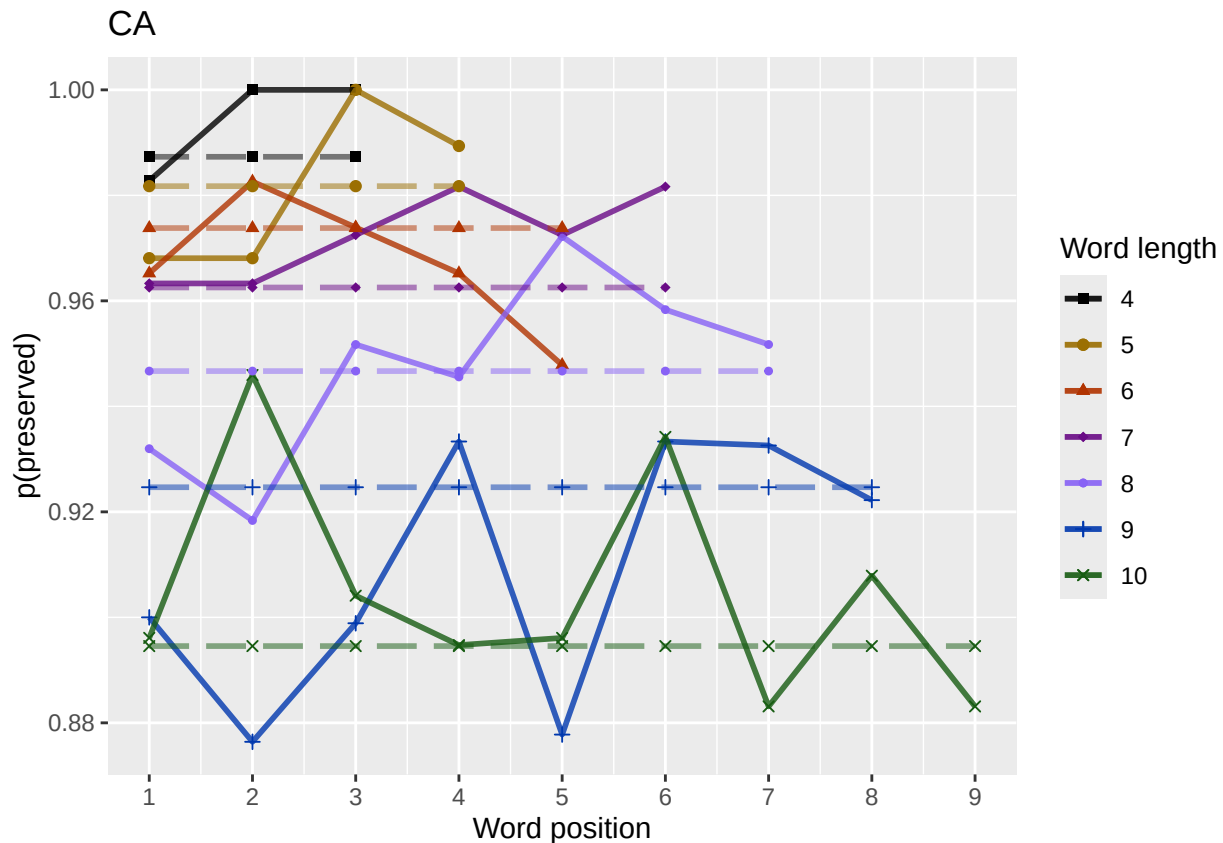
```
## 7     10 0.895 0.895 0.895 0.895 0.895 0.895 0.895 0.895 0.895
```

```
fitted_pos_len_summary$stimlen<-factor(fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary, aes(x=pos, y=preserved, group=stimlen, shape=stimlen, color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))
# geom_point(data=fitted_pos_len_summary, aes(x=pos, y=fitted, group=stimlen, shape=stimlen)) + ggtitle(paste0("Patient", patient_id))

fitted_len_pos_plot <- plot_len_pos_obs_predicted(PosDat,
  paste0(PosDat$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_", CurTask, "_percent_preserved_by_length_pos_wfit.png"), plot=fitted_len_pos_plot)
```



length and position without fragments to see if this changes position² influence

```
# first number responses, then count resp with fragments - below we will eliminate fragments
# and re-run models
```

```

# number responses
resp_num<-0
prev_pos<-9999 # big number to initialize (so first position is smaller)
resp_num_array <- integer(length = nrow(PosDat))
for(i in seq(1,nrow(PosDat))){
  if(PosDat$pos[i] <= prev_pos){ # equal for resp length 1
    resp_num <- resp_num + 1
  }
  resp_num_array[i]<-resp_num
  prev_pos <- PosDat$pos[i]
}
PosDat$resp_num <- resp_num_array

# count responses with fragments
resp_with_frag <- PosDat %>% group_by(resp_num) %>% summarise(frag = as.logical(sum(fragment)))
num_frag <- resp_with_frag %>% summarise(frag_sum = sum(frag), N=n())
num_frag

## # A tibble: 1 x 2
##   frag_sum      N
##   <int> <int>
## 1      8  690

num_frag$percent_with_frag[1] <- (num_frag$frag_sum[1] / num_frag$N[1])*100

## Warning: Unknown or uninitialised column: `percent_with_frag`.

print(sprintf("The number of responses with fragments was %d / %d = %.2f percent",
              num_frag$frag_sum[1],num_frag$N[1],num_frag$percent_with_frag[1]))

## [1] "The number of responses with fragments was 8 / 690 = 1.16 percent"

write.csv(num_frag,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_percent_fragments.csv"),row.names = FALSE)

# length and position models for data without fragments
NoFragData <- PosDat %>% filter(fragment != 1)

NoFrag_LPRes<-TestModels(LPModelEquations,NoFragData)

## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##   data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos
##    5.65924    -0.37968     0.09054
##
## Degrees of Freedom: 4174 Total (i.e. Null);  4172 Residual
## Null Deviance:      1641
## Residual Deviance: 1581 AIC: 1587
## log likelihood:  -790.293
## Nagelkerke R2:  0.04446299
## % pres/err predicted correctly:  -385.0934
## % of predictable range [ (model-null)/(1-null) ]:  0.0142415

```

```

## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)          pos
##      5.47148      -0.37418      -0.01067      0.18245
##
## Degrees of Freedom: 4174 Total (i.e. Null); 4171 Residual
## Null Deviance:      1641
## Residual Deviance: 1580 AIC: 1588
## log likelihood: -790.0188
## Nagelkerke R2: 0.04486121
## % pres/err predicted correctly: -385.0447
## % of predictable range [ (model-null)/(1-null) ]: 0.01436583
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos stimlen:pos
##      5.26020      -0.33403      0.22211      -0.01474
##
## Degrees of Freedom: 4174 Total (i.e. Null); 4171 Residual
## Null Deviance:      1641
## Residual Deviance: 1580 AIC: 1588
## log likelihood: -790.1337
## Nagelkerke R2: 0.04469435
## % pres/err predicted correctly: -385.0672
## % of predictable range [ (model-null)/(1-null) ]: 0.01430836
## *****
## model index: 8
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos stimlen:I(pos^2)      stimlen:I(pos^2)
##      4.566834      -0.270156      -0.087883      0.796538      0.008561      -0.008561
##
## Degrees of Freedom: 4174 Total (i.e. Null); 4169 Residual
## Null Deviance:      1641
## Residual Deviance: 1580 AIC: 1592
## log likelihood: -789.7583
## Nagelkerke R2: 0.0452395
## % pres/err predicted correctly: -385.0535
## % of predictable range [ (model-null)/(1-null) ]: 0.01434328
## *****
## model index: 2
##

```

```

## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      5.6739      -0.3382
##
## Degrees of Freedom: 4174 Total (i.e. Null);  4173 Residual
## Null Deviance:      1641
## Residual Deviance: 1588  AIC: 1592
## log likelihood:  -793.9369
## Nagelkerke R2:  0.03916607
## % pres/err predicted correctly:  -385.9045
## % of predictable range [ (model-null)/(1-null) ]:  0.01217067
## *****
## model index:  1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.958
##
## Degrees of Freedom: 4174 Total (i.e. Null);  4174 Residual
## Null Deviance:      1641
## Residual Deviance: 1641  AIC: 1643
## log likelihood:  -820.6848
## Nagelkerke R2:  -1.366145e-15
## % pres/err predicted correctly:  -390.6714
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
## model index:  6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.57898      -0.02839      0.24372
##
## Degrees of Freedom: 4174 Total (i.e. Null);  4172 Residual
## Null Deviance:      1641
## Residual Deviance: 1637  AIC: 1643
## log likelihood:  -818.7198
## Nagelkerke R2:  0.002894337
## % pres/err predicted correctly:  -390.2783
## % of predictable range [ (model-null)/(1-null) ]:  0.001003696
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##

```

```
## Coefficients:
## (Intercept)          pos
##      2.966365      -0.002095
##
## Degrees of Freedom: 4174 Total (i.e. Null);  4173 Residual
## Null Deviance:      1641
## Residual Deviance: 1641 AIC: 1645
## log likelihood:  -820.6828
## Nagelkerke R2:   2.863126e-06
## % pres/err predicted correctly:  -390.671
## % of predictable range [ (model-null)/(1-null) ]:  9.954182e-07
## *****
```

```
NoFragBestLPModel<-NoFrag_LPres$ModelResult[[1]]
NoFragBestLPModelFormula<-NoFrag_LPres$Model[[1]]

NoFragLPAICSummary<-data.frame(Model=NoFrag_LPres$Model,
                                AIC=NoFrag_LPres$AIC,
                                row.names = NoFrag_LPres$Model)
NoFragLPAICSummary$DeltaAIC<-NoFragLPAICSummary$AIC-NoFragLPAICSummary$AIC[1]
NoFragLPAICSummary$AICexp<-exp(-0.5*NoFragLPAICSummary$DeltaAIC)
NoFragLPAICSummary$AICwt<-NoFragLPAICSummary$AICexp/sum(NoFragLPAICSummary$AICexp)
NoFragLPAICSummary$NagR2<-NoFrag_LPres$NagR2

NoFragLPAICSummary <- merge(NoFragLPAICSummary,NoFrag_LPres$CoefficientValues, by='row.names',sort=FALSE)
NoFragLPAICSummary <- subset(NoFragLPAICSummary, select = -c(Row.names))

write.csv(NoFragLPAICSummary,paste0(TablesDir,
                                    CurPat,"_",RunLabel,"_",
                                    CurTask,
                                    "_no_fragments_len_pos_model_summary.csv"),
          row.names = FALSE)
kable(NoFragLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	stimlen	pos	stimlen:I(pos)	I(pos^2)	stimlen:I(pos^2)
preserved ~	1586.586	0.000000	1.000000	0.000000	0.482760	0.044463	0.0659236	-	0.0905364	NA	NA
stimlen + pos							0.3796763				
preserved ~	1588.038	1.451590	0.483938	0.723362	0.004486	0.12471475	-	0.1824503	NA	-	NA
stimlen + I(pos^2)							0.3741819			0.0106703	
+ pos											
preserved ~	1588.267	1.681398	0.431408	0.920826	0.004469	0.13260197	-	0.2221140	-	NA	NA
stimlen * pos							0.3340308		0.0147446		
preserved ~	1591.514	4.930550	0.084985	0.104102	0.004523	0.15566834	-	0.7965379	-	-	0.0085607
stimlen * (I(pos^2)							0.2701557		0.0692840	0.0878825	
+ pos)											
preserved ~	1591.874	5.287815	0.071088	0.003431	0.010391	0.1651673943	-	NA	NA	NA	NA
stimlen							0.3382484				
preserved ~ 1	1643.373	6.783561	0.000000	0.000000	0.000000	0.0058393	NA	NA	NA	NA	NA
preserved ~	1643.446	6.853605	0.000000	0.000000	0.000289	0.13578985	NA	0.2437205	NA	-	NA
I(pos^2) + pos										0.0283903	
preserved ~ pos	1645.368	8.779670	0.000000	0.000000	0.000000	0.2966365	NA	-	NA	NA	NA
								0.0020945			

```
# plot no fragment data
```

```
NoFragData$LPFitted<-fitted(NoFragBestLPModel)
nofrag_obs_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData, NoFragData$patient[1],
  NULL,palette_values=palette_values)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
# len/pos table
```

```
nofrag_fitted_pos_len_summary <- NoFragData %>% group_by(stimlen,pos) %>% summarise(fitted = mean(LPfit
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
nofrag_fitted_pos_len_table <- nofrag_fitted_pos_len_summary %>% pivot_wider(names_from = pos, values_f
nofrag_fitted_pos_len_table
```

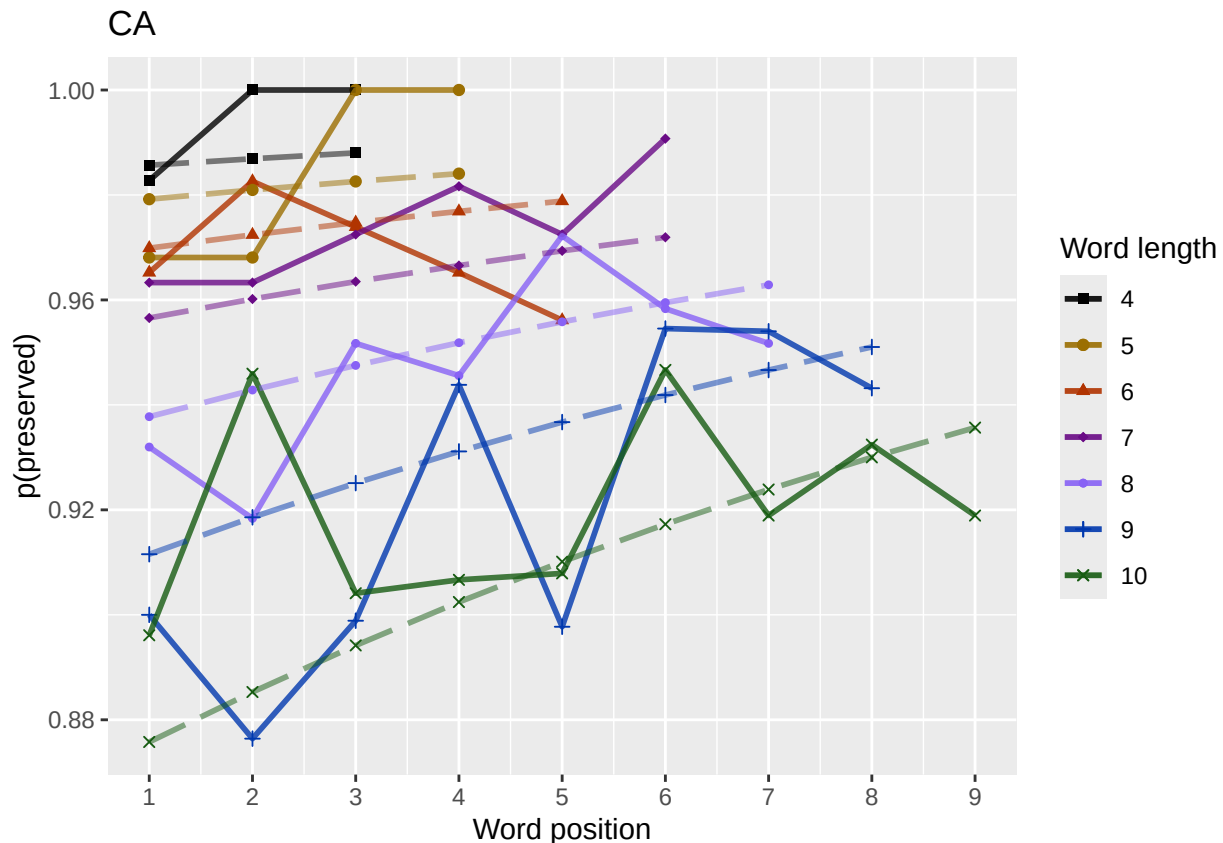
```
## # A tibble: 7 x 10
## # Groups:   stimlen [7]
##   stimlen   `1`   `2`   `3`   `4`   `5`   `6`   `7`   `8`   `9`
##   <int> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1      4 0.986 0.987 0.988 NA      NA      NA      NA      NA      NA
## 2      5 0.979 0.981 0.983 0.984 NA      NA      NA      NA      NA
## 3      6 0.970 0.972 0.975 0.977 0.979 NA      NA      NA      NA
## 4      7 0.957 0.960 0.963 0.967 0.969 0.972 NA      NA      NA
## 5      8 0.938 0.943 0.948 0.952 0.956 0.959 0.963 NA      NA
## 6      9 0.912 0.919 0.925 0.931 0.937 0.942 0.947 0.951 NA
## 7     10 0.876 0.885 0.894 0.902 0.910 0.917 0.924 0.930 0.936
```

```
fitted_pos_len_summary$stimlen<-factor(nofrag_fitted_pos_len_summary$stimlen)
fitted_pos_len_summary$pos<-factor(nofrag_fitted_pos_len_summary$pos)
# fitted_len_pos_plot <- ggplot(pos_len_summary,aes(x=pos,y=preserved,group=stimlen,shape=stimlen,color=stimlen))
# fitted_len_pos_plot <- fitted_len_pos_plot + geom_line(data=fitted_pos_len_summary, aes(x=pos,y=fitted,group=stimlen,shape=stimlen))
# geom_point(data=fitted_pos_len_summary,aes(x=pos,y=fitted,group=stimlen,shape=stimlen)) + ggtitle(paste0("Patient",patient[1]))
```

```
nofrag_fitted_len_pos_plot <- plot_len_pos_obs_predicted(NoFragData,
  paste0(NoFragData$patient[1]),
  "LPFitted",
  NULL,
  palette_values,
  shape_values,
  obs_linetypes,
  pred_linetypes = c("longdash")
)
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

```
ggsave(paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,
  "no_fragments_percent_preserved_by_length_pos_wfit.png"),
  plot=nofrag_fitted_len_pos_plot,
  device="png",unit="cm",
  width=15,height=11)
nofrag_fitted_len_pos_plot
```

back to full data

```
results_report_DF <- AddReportLine(results_report_DF,"min preserved",min_preserved)
results_report_DF <- AddReportLine(results_report_DF,"max preserved",max_preserved)
print(sprintf("Min/max preserved range: %.2f - %.2f",min_preserved,max_preserved))
```

```
## [1] "Min/max preserved range: 0.86 - 1.01"
```

```
# find the average difference between lengths and the average difference
# between positions taking the other factor into account
```

```
# choose fitted or raw -- we use fitted values because they are smoothed averages
# that take into account the influence of the other variable
```

```
table_to_use <- fitted_pos_len_table
# take away column with length information
table_to_use <- table_to_use[,2:ncol(table_to_use)]
```

```
# take averages for positions
```

```
# don't want downward estimates influenced by return upward of U
```

```
# therefore, for downward influence, use only the values before the min
```

```
# take the difference between each value (differences between position proportion correct) **NOTE** pro
```

```
# average the difference in probabilities
```

```
# in case min is close to the end or we are not using a min (for non-U-shaped)
```

```
# functions, we need to get differences _first_ and then average those (e.g.
```

```
# if there is an effect of length and we just average position data,
```

```
# later positions) will have a lower average (because of the length effect)
```

```
# even if all position functions go upward
```

```

table_pos_diffs <- t(diff(t(as.matrix(table_to_use))))
ncol_pos_diff_matrix <- ncol(table_pos_diffs)
first_col_mean <- mean(as.numeric(unlist(table_pos_diffs[,1])))
potential_u_shape <- 0
if(first_col_mean < 0){ # downward, so potential U-shape
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs >= 0] <- NA
  potential_u_shape <- 1
}else{
  # upward - use the positive values
  # take only negative values from the table
  filtered_table_pos_diffs<-table_pos_diffs
  filtered_table_pos_diffs[table_pos_diffs <= 0] <- NA
}
average_pos_diffs <- apply(filtered_table_pos_diffs,2,mean,na.rm=TRUE)
OA_mean_pos_diff <- mean(average_pos_diffs,na.rm=TRUE)

table_len_diffs <- diff(as.matrix(table_to_use))
average_len_diffs <- apply(table_len_diffs,1,mean,na.rm=TRUE)
OA_mean_len_diff <- mean(average_len_diffs,na.rm=TRUE)
CurrentLabel<-"mean change in probability for each additional length: "
print(CurrentLabel)

## [1] "mean change in probability for each additional length: "
print(OA_mean_len_diff)

## [1] -0.01545941
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_len_diff)

CurrentLabel<-"mean change in probability for each additional position (excluding U-shape): "
print(CurrentLabel)

## [1] "mean change in probability for each additional position (excluding U-shape): "
print(OA_mean_pos_diff)

## [1] NaN
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,OA_mean_pos_diff)

if(potential_u_shape){ # downward, so potential U-shape
  # take only positive values from the table
  filtered_pos_upward_u<-table_pos_diffs
  filtered_pos_upward_u[table_pos_diffs <= 0] <- NA
  average_pos_u_diffs <- apply(filtered_pos_upward_u,
    2,mean,na.rm=TRUE)
  OA_mean_pos_u_diff <- mean(average_pos_u_diffs,na.rm=TRUE)
  if(is.nan(OA_mean_pos_u_diff) | (OA_mean_pos_u_diff <= 0)){
    print("No U-shape in this participant")
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
    potential_u_shape <- FALSE
  }else{
    results_report_DF <- AddReportLine(results_report_DF, "U-shape", 1)
  }
}

```

```

    CurrentLabel<-"Average upward change after U minimum"
    print(CurrentLabel)
    print(OA_mean_pos_u_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, OA_mean_pos_u_diff)

    CurrentLabel<-"Proportion of average downward change"
    prop_ave_down <- abs(OA_mean_pos_u_diff/OA_mean_pos_diff)
    results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, prop_ave_down)
  }
}else{
  print("No U-shape in this participant")
  results_report_DF <- AddReportLine(results_report_DF, "U-shape", 0)
}

## [1] "No U-shape in this participant"
if(potential_u_shape){
  n_rows<-nrow(table_to_use)
  downward_dist <- numeric(n_rows)
  upward_dist <- numeric(n_rows)
  for(i in seq(1,n_rows)){
    current_row <- as.numeric(unlist(table_to_use[i,1:9]))
    current_row <- current_row[!is.na(current_row)]
    current_row_len <- length(current_row)
    row_min <- min(current_row,na.rm=TRUE)
    min_pos <- which(current_row == row_min)
    left_max <- max(current_row[1:min_pos],na.rm=TRUE)
    right_max <- max(current_row[min_pos:current_row_len])
    left_diff <- left_max - row_min
    right_diff <- right_max - row_min
    downward_dist[i]<-left_diff
    upward_dist[i]<-right_diff
  }
  print("differences from left max to min for each row: ")
  print(downward_dist)
  print("differences from min to right max for each row: ")
  print(upward_dist)

  # Use the max return upward (min of upward_dist) and then the
  # downward distance from the left for the same row

  print(" ")
  biggest_return_upward_row <- which(upward_dist == max(upward_dist))
  CurrentLabel <- "row with biggest return upward"
  print(CurrentLabel)
  print(biggest_return_upward_row)
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, biggest_return_upward_row)

  print(" ")
  CurrentLabel<-"downward distance for row with the largest upward value"
  print(CurrentLabel)
  print(downward_dist[biggest_return_upward_row])
  results_report_DF <- AddReportLine(results_report_DF, CurrentLabel, downward_dist[biggest_return_upward_row])
}

```

```

CurrentLabel<-"return upward value"
print(upward_dist[biggest_return_upward_row])
results_report_DF <- AddReportLine(results_report_DF,
                                   CurrentLabel,
                                   upward_dist[biggest_return_upward_row])

print(" ")
# percentage return upward
percentage_return_upward <- upward_dist[biggest_return_upward_row]/downward_dist[biggest_return_upward_row]
CurrentLabel <- "Return upward as a proportion of the downward distance:"
print(CurrentLabel)
print(percentange_return_upward)
results_report_DF <- AddReportLine(results_report_DF, CurrentLabel,
                                   percentange_return_upward)
}else{
  print("no U-shape in this participant")
}

## [1] "no U-shape in this participant"

FLPModelEquations<-c(
  # models with log frequency, but no three-way interactions (due to difficulty interpreting and small
  "preserved ~ stimlen*log_freq",
  "preserved ~ stimlen+log_freq",
  "preserved ~ pos*log_freq",
  "preserved ~ pos+log_freq",
  "preserved ~ stimlen*log_freq + pos*log_freq",
  "preserved ~ stimlen*log_freq + pos",
  "preserved ~ stimlen + pos*log_freq",
  "preserved ~ stimlen + pos + log_freq",
  "preserved ~ (I(pos^2)+pos)*log_freq",
  "preserved ~ stimlen*log_freq + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen*log_freq + I(pos^2) + pos",
  "preserved ~ stimlen + (I(pos^2) + pos)*log_freq",
  "preserved ~ stimlen + I(pos^2) + pos + log_freq",

  # models without frequency
  "preserved ~ 1",
  "preserved ~ stimlen",
  "preserved ~ pos",
  "preserved ~ stimlen + pos",
  "preserved ~ stimlen*pos",
  "preserved ~ I(pos^2)+pos",
  "preserved ~ stimlen + I(pos^2) + pos",
  "preserved ~ stimlen * (I(pos^2) + pos)"
)

FLPRes<-TestModels(FLPModelEquations,PosDat)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##

```

```

## Coefficients:
## (Intercept)      stimlen      log_freq
##      5.4628      -0.3133      0.1939
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1681 AIC: 1687
## log likelihood:  -840.5675
## Nagelkerke R2:  0.06569778
## % pres/err predicted correctly:  -421.9769
## % of predictable range [ (model-null)/(1-null) ]:  0.02318462
## *****
## model index:  8
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos      log_freq
##      5.46220      -0.33249      0.03919      0.19404
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1680 AIC: 1688
## log likelihood:  -839.8028
## Nagelkerke R2:  0.06672904
## % pres/err predicted correctly:  -421.7986
## % of predictable range [ (model-null)/(1-null) ]:  0.02359643
## *****
## model index:  1
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      log_freq stimlen:log_freq
##      5.54501      -0.32456      0.42051      -0.02676
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1680 AIC: 1688
## log likelihood:  -840.0397
## Nagelkerke R2:  0.06640965
## % pres/err predicted correctly:  -422.0838
## % of predictable range [ (model-null)/(1-null) ]:  0.02293779
## *****
## model index:  13
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos      log_freq
##      5.16023      -0.32306      -0.01579      0.17883      0.19444

```

```

##
## Degrees of Freedom: 4197 Total (i.e. Null); 4193 Residual
## Null Deviance: 1777
## Residual Deviance: 1678 AIC: 1688
## log likelihood: -839.1037
## Nagelkerke R2: 0.06767153
## % pres/err predicted correctly: -421.6067
## % of predictable range [ (model-null)/(1-null) ]: 0.02403946
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq
## 5.54405 -0.34361 0.41977 0.03905 -0.02666
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4193 Residual
## Null Deviance: 1777
## Residual Deviance: 1679 AIC: 1689
## log likelihood: -839.2795
## Nagelkerke R2: 0.06743467
## % pres/err predicted correctly: -421.9001
## % of predictable range [ (model-null)/(1-null) ]: 0.0233619
## *****
## model index: 11
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stimlen:log
## 5.23376 -0.33416 0.42918 -0.01635 0.18358 -0
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4192 Residual
## Null Deviance: 1777
## Residual Deviance: 1677 AIC: 1689
## log likelihood: -838.5313
## Nagelkerke R2: 0.06844306
## % pres/err predicted correctly: -421.7079
## % of predictable range [ (model-null)/(1-null) ]: 0.02380584
## *****
## model index: 7
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen pos log_freq pos:log_freq
## 5.47723 -0.32941 0.02857 0.23970 -0.01115
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4193 Residual
## Null Deviance: 1777

```

```

## Residual Deviance: 1679 AIC: 1689
## log likelihood: -839.5691
## Nagelkerke R2: 0.06704416
## % pres/err predicted correctly: -421.7459
## % of predictable range [ (model-null)/(1-null) ]: 0.02371811
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq pos stimlen:log_freq log_fr
## 5.544822 -0.340644 0.420471 0.032753 -0.023454 -0.
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4192 Residual
## Null Deviance: 1777
## Residual Deviance: 1678 AIC: 1690
## log likelihood: -839.2005
## Nagelkerke R2: 0.06754117
## % pres/err predicted correctly: -421.8541
## % of predictable range [ (model-null)/(1-null) ]: 0.02346814
## *****
## model index: 12
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen I(pos^2) pos log_freq I(pos
## 5.116148 -0.318475 -0.019343 0.198753 0.194379
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4191 Residual
## Null Deviance: 1777
## Residual Deviance: 1677 AIC: 1691
## log likelihood: -838.6936
## Nagelkerke R2: 0.06822438
## % pres/err predicted correctly: -421.5755
## % of predictable range [ (model-null)/(1-null) ]: 0.02411151
## *****
## model index: 10
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen log_freq I(pos^2) pos stim
## 5.190050 -0.329375 0.378537 -0.018724 0.197456
## log_freq:pos
## 0.015032
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4190 Residual
## Null Deviance: 1777
## Residual Deviance: 1677 AIC: 1693

```

```

## log likelihood: -838.3672
## Nagelkerke R2: 0.06866421
## % pres/err predicted correctly: -421.6712
## % of predictable range [ (model-null)/(1-null) ]: 0.02389053
## *****
## model index: 15
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      5.8316      -0.3693
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1708 AIC: 1712
## log likelihood: -854.1651
## Nagelkerke R2: 0.04729663
## % pres/err predicted correctly: -425.1153
## % of predictable range [ (model-null)/(1-null) ]: 0.01593689
## *****
## model index: 17
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      pos
##      5.83090      -0.38827      0.03868
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1707 AIC: 1713
## log likelihood: -853.4134
## Nagelkerke R2: 0.04831697
## % pres/err predicted correctly: -424.9824
## % of predictable range [ (model-null)/(1-null) ]: 0.01624368
## *****
## model index: 20
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      I(pos^2)      pos
##      5.54224      -0.37944      -0.01515      0.17281
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1706 AIC: 1714
## log likelihood: -852.761
## Nagelkerke R2: 0.0492022
## % pres/err predicted correctly: -424.808

```



```

## % of predictable range [ (model-null)/(1-null) ]: 0.01664653
## *****
## model index: 18
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen          pos  stimlen:pos
##      5.1397      -0.3095      0.2568      -0.0243
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1706 AIC: 1714
## log likelihood: -852.921
## Nagelkerke R2: 0.04898515
## % pres/err predicted correctly: -424.811
## % of predictable range [ (model-null)/(1-null) ]: 0.01663952
## *****
## model index: 21
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      stimlen      I(pos^2)          pos  stimlen:I(pos^2)      stimlen:I(pos^2)
##      4.725517      -0.281959      -0.062484      0.635549      0.005506      -0.005506
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4192 Residual
## Null Deviance:      1777
## Residual Deviance: 1705 AIC: 1717
## log likelihood: -852.5377
## Nagelkerke R2: 0.04950514
## % pres/err predicted correctly: -424.782
## % of predictable range [ (model-null)/(1-null) ]: 0.01670642
## *****
## model index: 9
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
##      (Intercept)      I(pos^2)          pos      log_freq  I(pos^2):log_freq
##      2.670620      -0.034915      0.253604      0.282295      -0.004484
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4192 Residual
## Null Deviance:      1777
## Residual Deviance: 1716 AIC: 1728
## log likelihood: -858.2174
## Nagelkerke R2: 0.04178964
## % pres/err predicted correctly: -425.5056
## % of predictable range [ (model-null)/(1-null) ]: 0.01503538
## *****
## model index: 4

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq
##      3.09703      -0.03517      0.25407
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1724 AIC: 1730
## log likelihood:  -862.1159
## Nagelkerke R2:  0.03648177
## % pres/err predicted correctly:  -426.415
## % of predictable range [ (model-null)/(1-null) ]:  0.01293533
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos      log_freq pos:log_freq
##      3.17824      -0.05691      0.34665      -0.02297
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1722 AIC: 1730
## log likelihood:  -861.1812
## Nagelkerke R2:  0.03775518
## % pres/err predicted correctly:  -426.1679
## % of predictable range [ (model-null)/(1-null) ]:  0.01350595
## *****
## model index:  19
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.6091      -0.0329      0.2332
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1767 AIC: 1773
## log likelihood:  -883.7294
## Nagelkerke R2:  0.006874509
## % pres/err predicted correctly:  -430.8122
## % of predictable range [ (model-null)/(1-null) ]:  0.002780514
## *****
## model index:  16
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)

```

```
##
## Coefficients:
## (Intercept)          pos
##      3.09117      -0.06055
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1774  AIC: 1778
## log likelihood:  -886.8559
## Nagelkerke R2:  0.002566396
## % pres/err predicted correctly:  -431.5924
## % of predictable range [ (model-null)/(1-null) ]:  0.0009788256
## *****
## model index:  14
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.853
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4197 Residual
## Null Deviance:      1777
## Residual Deviance: 1777  AIC: 1779
## log likelihood:  -888.7162
## Nagelkerke R2:  3.21633e-16
## % pres/err predicted correctly:  -432.0162
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
BestFLPModel<-FLPRes$ModelResult[[1]]
BestFLPModelFormula<-FLPRes$Model[[1]]

FLPAICSummary<-data.frame(Model=FLPRes$Model,
                           AIC=FLPRes$AIC,row.names=FLPRes$Model)
FLPAICSummary$DeltaAIC<-FLPAICSummary$AIC-FLPAICSummary$AIC[1]
FLPAICSummary$AICexp<-exp(-0.5*FLPAICSummary$DeltaAIC)
FLPAICSummary$AICwt<-FLPAICSummary$AICexp/sum(FLPAICSummary$AICexp)
FLPAICSummary$NagR2<-FLPRes$NagR2

FLPAICSummary <- merge(FLPAICSummary,FLPRes$CoefficientValues,
                       by='row.names',sort=FALSE)
FLPAICSummary <- subset(FLPAICSummary, select = -c(Row.names))

write.csv(FLPAICSummary,paste0(TablesDir,CurPat,"_",
                               RunLabel,"_",CurTask,
                               "_freq_len_pos_model_summary.csv"),row.names = FALSE)

kable(FLPAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	log_freq	log_stimlen	log_pos	log_freq_pos	log_freq_pos^2	log_freq_pos^3	log_freq_pos^4	log_freq_pos^5	len:I(pos^2)
preserved ~	1687.135	0.000000	1.000000	0.1074155	0.9782782	0.1938801	NA	NA	NA	NA	NA	NA	NA	NA	NA
stimlen +						0.3133170									
log_freq															

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_freq	log_pos	log_freq(I(pos^2))	pos^2	log_freq(I(pos^2))	pos	log_freq(I(pos^2))	len:I(pos^2)
preserved ~ stimlen + pos + log_freq	1687.667	00667908	01792925	2462	205	0.194035	0.039013	NA	NA	NA	NA	NA	NA	NA
preserved ~ stimlen *	1688.074	00772363	3506763	1945	011	0.4205055	NA	NA	NA	NA	NA	NA	NA	NA
log_freq						0.3245587	0.0267611							
preserved ~ stimlen + I(pos^2) + pos + log_freq	1688.207	00984940	2678007	5716	225	0.1944361	0.1788339	NA	-	NA	NA	NA	NA	NA
						0.3230625				0.0157923				
preserved ~ stimlen *	1688.532	00140008	2663512	1347	049	0.4197654	0.0390169	NA	NA	NA	NA	NA	NA	NA
log_freq + pos						0.3436140	0.0266575							
preserved ~ stimlen *	1689.162	00758161	4082676	8143	759	0.4291844	0.1835811	NA	-	NA	NA	NA	NA	NA
log_freq + I(pos^2) + pos						0.3341649	0.0277217			0.0163520				
preserved ~ stimlen + pos * log_freq	1689.238	00336728	7960679	4472	227	0.2397015	0.0285659	NA	NA	NA	NA	NA	NA	NA
						0.3294052		0.0111549						
preserved ~ stimlen *	1690.302	00590715	3314234	0875	512	0.4204710	0.0327526	-	NA	NA	NA	NA	NA	NA
log_freq + pos *						0.3406442	0.0234538			0.0068018				
log_freq														
preserved ~ stimlen + (I(pos^2) + pos) *	1691.425	00301930	0585882	2446	148	0.1943791	0.1980525	NA	-	-	NA	NA	NA	NA
log_freq						0.3184747				0.0196403				
preserved ~ stimlen *	1692.534	00933508	2173086	5610	050	0.3785371	0.1974558	0.0150324	NA	-	NA	NA	NA	NA
log_freq + (I(pos^2) + pos) *						0.3293749	0.0223314			0.0187238		0.0025342		
log_freq														
preserved ~ stimlen	1712.230	00951070	0000000	7293	155	NA	NA	NA	NA	NA	NA	NA	NA	NA
						0.3693428								
preserved ~ stimlen + pos	1712.227	00801900	0260000	1838	899	NA	NA	0.0386765	NA	NA	NA	NA	NA	NA
						0.3882686								
preserved ~ stimlen + I(pos^2) + pos	1713.222	00800700	0000000	1922	242	NA	NA	0.1728132	NA	-	NA	NA	NA	NA
						0.3794447					0.0151525			
preserved ~ stimlen * pos	1713.261	00029100	0000000	1893	682	NA	NA	0.2567670	NA	NA	NA	NA	-	NA
						0.3095467							0.0243008	

Model	AIC Delta	AIC	AICw	NagR ²	Intercept	log_stimlen	log_pos	log_freq	I(pos^2)	log_freq:I(pos^2)	log_freq:I(pos)	log_freq:I(pos^2)	len:I(pos^2)
preserved ~ stimlen * (I(pos^2) + pos)	1717.2754	4090.0000	0.0000	0.4517	5.4628	NA	NA	0.6355	NA	-	NA	NA	- 0.0055065
				0.2819	5.588					0.0624	842		0.0544492
preserved ~ (I(pos^2) + pos) * log_freq	1728.4329	3989.0000	0.0000	0.4722	5.628	0.2822	0.48	0.2536	0.16	NA	-	- NA	NA NA
										0.0349	106		44841
preserved ~ pos + log_freq	1730.4329	3967.0000	0.0000	0.3643	0.8702	0.2540	0.64	-	NA	NA	NA	NA	NA NA
								0.0351	748				
preserved ~ pos * log_freq	1730.4322	3947.0000	0.0000	0.3735	0.8214	0.3460	0.85	-	- NA	NA	NA	NA	NA NA
								0.0569	0.82	9660			
preserved ~ I(pos^2) + pos	1773.8632	3981.0000	0.0000	0.6870	NA	NA	NA	0.2337	NA	-	NA	NA	NA NA
										0.0328	963		
preserved ~ pos	1777.9123	3767.0000	0.0000	0.2369	1.72	NA	NA	-	NA	NA	NA	NA	NA NA
								0.0605	511				
preserved ~ 1	1779.9229	3703.0000	0.0000	0.0000	0.8525	NA	NA	NA	NA	NA	NA	NA	NA NA

```
print(BestFLPModelFormula)
```

```
## [1] "preserved ~ stimlen + log_freq"
```

```
print(BestFLPModel)
```

```
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen      log_freq
##      5.4628      -0.3133      0.1939
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1681 AIC: 1687
```

```
# do a median split on frequency to plot hf/ls effects (analysis is continuous)
median_freq <- median(PosDat$log_freq)
PosDat$freq_bin[PosDat$log_freq >= median_freq] <- "hf"
PosDat$freq_bin[PosDat$log_freq < median_freq] <- "lf"
```

```
PosDat$FLPFitted <- fitted(BestFLPModel)
```

```
HFDat <- PosDat[PosDat$freq_bin == "hf",]
LFDat <- PosDat[PosDat$freq_bin == "lf",]
```

```
HF_Plot <- plot_len_pos_obs_predicted(HFDat, paste0(CurPat, " - ", RunLabel, " - High frequency"), "FLPFitted")
```

```
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
```

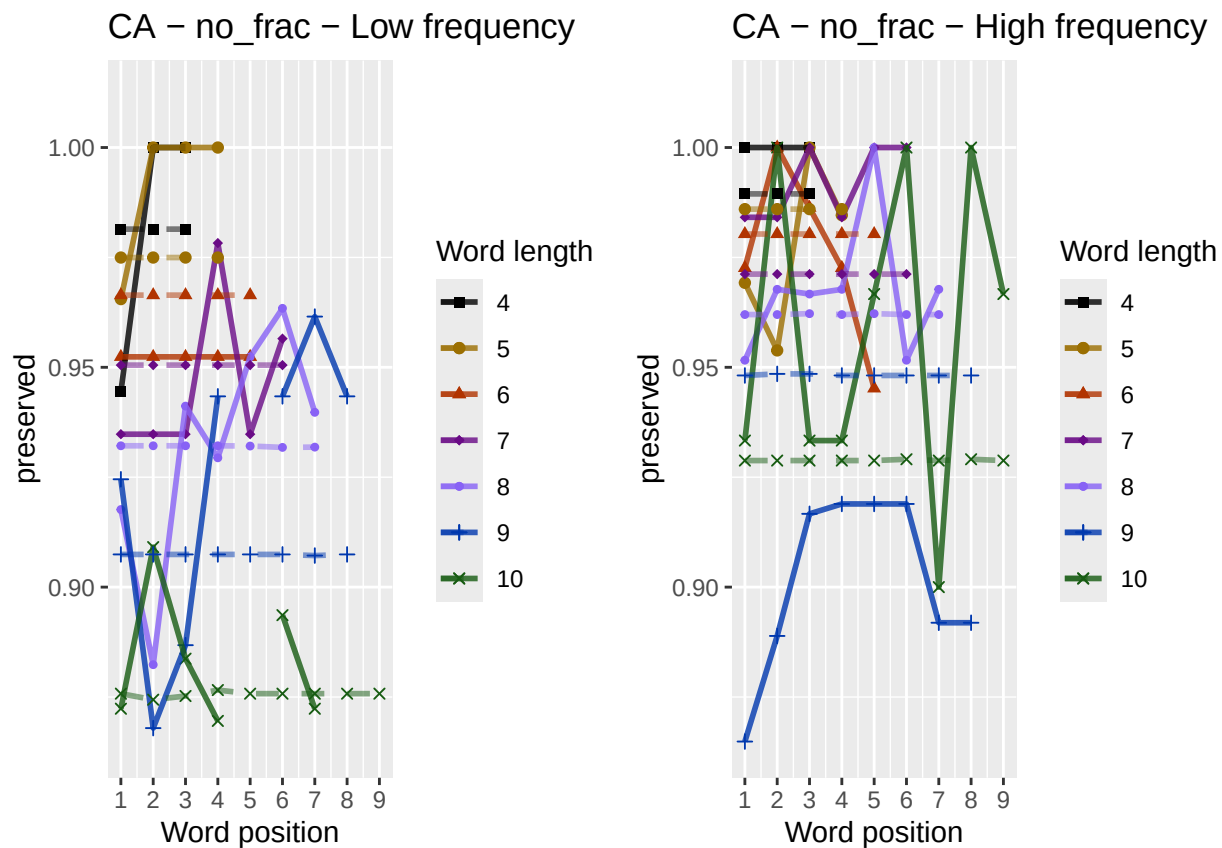
```
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.
LF_Plot <- plot_len_pos_obs_predicted(LFdat, paste0(CurPat, " - ", RunLabel, " - Low frequency"), "FLPFitted")

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## Scale for y is already present. Adding another scale for y, which will replace the existing scale.

library(ggpubr)
Both_Plots <- ggarrange(LF_Plot, HF_Plot) # labels=c("LF", "HF", ncol=2)

## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: Removed 2 rows containing missing values or values outside the scale range (`geom_line()`).

ggsave(paste0(FigDir, CurPat, "_", RunLabel, "_",
              CurTask, "_frequency_effect_length_pos_wfit.png"),
       device="png", unit="cm", width=30, height=11)
print(Both_Plots)
```



```
# only main effects
MEModelEquations<-c(
  "preserved ~ CumPres",
  "preserved ~ CumErr",
  "preserved ~ (I(pos^2)+pos)",
  "preserved ~ pos",
  "preserved ~ stimlen",
  "preserved ~ 1"
)
```

```
MERes<-TestModels(MEModelEquations,PosDat)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.417      -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1418 AIC: 1422
## log likelihood: -709.0299
## Nagelkerke R2: 0.237682
## % pres/err predicted correctly: -340.4165
## % of predictable range [ (model-null)/(1-null) ]: 0.2115386
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      5.8316      -0.3693
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1708 AIC: 1712
## log likelihood: -854.1651
## Nagelkerke R2: 0.04729663
## % pres/err predicted correctly: -425.1153
## % of predictable range [ (model-null)/(1-null) ]: 0.01593689
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.2255      0.2967
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1714 AIC: 1718
## log likelihood: -857.243
## Nagelkerke R2: 0.04311484
## % pres/err predicted correctly: -425.239
## % of predictable range [ (model-null)/(1-null) ]: 0.01565124
## *****
```

```

## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)          pos
##      2.6091      -0.0329      0.2332
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1767 AIC: 1773
## log likelihood: -883.7294
## Nagelkerke R2: 0.006874509
## % pres/err predicted correctly: -430.8122
## % of predictable range [ (model-null)/(1-null) ]: 0.002780514
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.09117      -0.06055
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1774 AIC: 1778
## log likelihood: -886.8559
## Nagelkerke R2: 0.002566396
## % pres/err predicted correctly: -431.5924
## % of predictable range [ (model-null)/(1-null) ]: 0.0009788256
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.853
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4197 Residual
## Null Deviance:      1777
## Residual Deviance: 1777 AIC: 1779
## log likelihood: -888.7162
## Nagelkerke R2: 3.21633e-16
## % pres/err predicted correctly: -432.0162
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
BestMEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
BestMEModelFormula<-MERes$Model[[BestModelIndexL1]]

```



```

MEAICSummary<-data.frame(Model=MERes$Model,
                          AIC=MERes$AIC,row.names=MERes$Model)
MEAICSummary$DeltaAIC<-MEAICSummary$AIC-MEAICSummary$AIC[1]
MEAICSummary$AICexp<-exp(-0.5*MEAICSummary$DeltaAIC)
MEAICSummary$AICwt<-MEAICSummary$AICexp/sum(MEAICSummary$AICexp)
MEAICSummary$NagR2<-MERes$NagR2

MEAICSummary <- merge(MEAICSummary,MERes$CoefficientValues,
                      by='row.names',sort=FALSE)
MEAICSummary <- subset(MEAICSummary, select = -c(Row.names))

write.csv(MEAICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                             CurTask,"_main_effects_model_summary.csv"),
          row.names = FALSE)
kable(MEAICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErrI(pos^2)	pos	stimlen
preserved ~ CumErr	1422.0600	0.0000	1	1	0.2376820	0.416626	NA	- 1.33392	NA	NA
preserved ~ stimlen	1712.3309	290.2704	0	0	0.0472966	0.831551	NA	NA	NA	- 0.3693428
preserved ~ CumPres	1718.4809	296.4262	0	0	0.0431148	0.225525	0.2966624	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1773.4595	351.3990	0	0	0.0068742	0.609117	NA	NA	- 0.2331777	NA
preserved ~ pos	1777.7123	355.6520	0	0	0.0025663	0.091172	NA	NA	NA	- 0.0605511
preserved ~ 1	1779.4323	357.3726	0	0	0.0000000	0.852547	NA	NA	NA	NA

```

if(DoSimulations){
  BestMEModelFormulaRnd <- BestMEModelFormula
  if(grepl("CumPres",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumPres","RndCumPres",BestMEModelFormulaRnd)
  }else if(grepl("CumErr",BestMEModelFormulaRnd)){
    BestMEModelFormulaRnd <- gsub("CumErr","RndCumErr",BestMEModelFormulaRnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestMEModelFormulaRnd),
                       family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
  ModelNames<-c(paste0("***",BestMEModelFormula),
                rep(BestMEModelFormulaRnd,RandomSamples))
  AICValues <- c(BestMEModel$aic,RndModelAIC)
  BestMEModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
  BestMEModelRndDF <- BestMEModelRndDF %>% arrange(AIC)
}

```

```

BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random average"),
                                       AIC=c(mean(RndModelAIC))))
BestMEMModelRndDF <- rbind(BestMEMModelRndDF,
                           data.frame(Name=c("Random SD"),
                                       AIC=c(sd(RndModelAIC))))

write.csv(BestMEMModelRndDF,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_best_main_effects_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

syll_component_summary <- PosDat %>%
  group_by(syll_component) %>% summarise(MeanPres = mean(preserved),
                                         N = n())
write.csv(syll_component_summary, paste0(TablesDir, CurPat, "_",
                                         RunLabel, "_", CurTask,
                                         "_syllable_component_summary.csv"), row.names = FALSE)
kable(syll_component_summary)

```

syll_component	MeanPres	N
1	0.9605027	557
O	0.9432697	1939
P	0.9411765	34
S	0.8975410	244
V	0.9508427	1424

```

# main effects models for data without satellite positions

keep_components = c("0", "V", "1")
OV1Data <- PosDat[PosDat$syll_component %in% keep_components,]
OV1Data <- OV1Data %>% select(stim_number,
                             stimlen, stim, pos,
                             preserved, syll_component)
OV1Data$CumPres <- CalcCumPres(OV1Data)
OV1Data$CumErr <- CalcCumErrFromPreserved(OV1Data)

SimpSyllMEAICSummary <- EvaluateSubsetData(OV1Data, MEModelEquations)

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.501      -1.499
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3918 Residual
## Null Deviance:      1592
## Residual Deviance: 1239 AIC: 1243

```

```

## log likelihood: -619.3969
## Nagelkerke R2: 0.2578684
## % pres/err predicted correctly: -293.3624
## % of predictable range [ (model-null)/(1-null) ]: 0.231794
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) stimlen
## 6.0419 -0.3869
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3918 Residual
## Null Deviance: 1592
## Residual Deviance: 1525 AIC: 1529
## log likelihood: -762.6784
## Nagelkerke R2: 0.05014295
## % pres/err predicted correctly: -375.8025
## % of predictable range [ (model-null)/(1-null) ]: 0.01664769
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumPres
## 2.2890 0.3181
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3918 Residual
## Null Deviance: 1592
## Residual Deviance: 1534 AIC: 1538
## log likelihood: -767.0122
## Nagelkerke R2: 0.04362029
## % pres/err predicted correctly: -376.233
## % of predictable range [ (model-null)/(1-null) ]: 0.01552442
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) I(pos^2) pos
## 2.73707 -0.03215 0.21290
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3917 Residual
## Null Deviance: 1592
## Residual Deviance: 1581 AIC: 1587
## log likelihood: -790.4406
## Nagelkerke R2: 0.008108074
## % pres/err predicted correctly: -380.9723

```

```
## % of predictable range [ (model-null)/(1-null) ]: 0.003156127
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)          pos
##      3.21736      -0.07621
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3918 Residual
## Null Deviance: 1592
## Residual Deviance: 1586 AIC: 1590
## log likelihood: -793.1223
## Nagelkerke R2: 0.00401597
## % pres/err predicted correctly: -381.612
## % of predictable range [ (model-null)/(1-null) ]: 0.001486535
## *****
## model index: 6
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.913
##
## Degrees of Freedom: 3919 Total (i.e. Null); 3919 Residual
## Null Deviance: 1592
## Residual Deviance: 1592 AIC: 1594
## log likelihood: -795.7506
## Nagelkerke R2: 9.981427e-16
## % pres/err predicted correctly: -382.1816
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****
```

```
write.csv(SimpSyllMEAICSummary,
  paste0(TablesDir, CurPat, "_", RunLabel, "_",
    CurTask,
    "_OV1_main_effects_model_summary.csv"),
  row.names = FALSE)
kable(SimpSyllMEAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1242.794	0.0000	1	1	0.2578683	1.500813	NA	-	NA	NA	NA
preserved ~ stimlen	1529.357	286.5630	0	0	0.0501420	0.041931	NA	NA	NA	NA	-
preserved ~ CumPres	1538.024	295.2305	0	0	0.0436203	0.289020	0.3180599	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1586.883	344.0873	0	0	0.0081082	0.737068	NA	NA	-	0.2128987	NA
									0.0321482		

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ pos	1590.243	47.4508	0	0	0.004016	0.217356	NA	NA	NA	-	NA
										0.0762100	
preserved ~ 1	1593.503	50.7074	0	0	0.000000	0.912673	NA	NA	NA	NA	NA

```
# main effects models for data without satellite or coda positions
# (tradeoff -- takes away more complex segments, in addition to satellites, but
# also reduces data)
```

```
keep_components = c("0","V")
OVDData <- PosDat[PosDat$syll_component %in% keep_components,]
OVDData <- OVDData %>% select(stim_number,
                             stimlen,stim,pos,
                             preserved,syll_component)
OVDData$CumPres <- CalcCumPres(OVDData)
OVDData$CumErr <- CalcCumErrFromPreserved(OVDData)

SimpSyllMEAICSummary2<-EvaluateSubsetData(OVDData,MEModelEquations)
```

```
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.407      -1.624
##
## Degrees of Freedom: 3362 Total (i.e. Null); 3361 Residual
## Null Deviance:      1404
## Residual Deviance: 1130 AIC: 1134
## log likelihood: -564.8471
## Nagelkerke R2: 0.2295916
## % pres/err predicted correctly: -269.5083
## % of predictable range [ (model-null)/(1-null) ]: 0.2060954
## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      stimlen
##      5.9083      -0.3765
##
## Degrees of Freedom: 3362 Total (i.e. Null); 3361 Residual
## Null Deviance:      1404
## Residual Deviance: 1347 AIC: 1351
## log likelihood: -673.6822
## Nagelkerke R2: 0.04904325
## % pres/err predicted correctly: -334.0111
## % of predictable range [ (model-null)/(1-null) ]: 0.01678862
```

```

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.295      0.353
##
## Degrees of Freedom: 3362 Total (i.e. Null); 3361 Residual
## Null Deviance:      1404
## Residual Deviance: 1357 AIC: 1361
## log likelihood: -678.3663
## Nagelkerke R2: 0.04100745
## % pres/err predicted correctly: -334.4845
## % of predictable range [ (model-null)/(1-null) ]: 0.01539913
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.63665      -0.03133      0.22542
##
## Degrees of Freedom: 3362 Total (i.e. Null); 3360 Residual
## Null Deviance:      1404
## Residual Deviance: 1397 AIC: 1403
## log likelihood: -698.5576
## Nagelkerke R2: 0.006110539
## % pres/err predicted correctly: -338.9027
## % of predictable range [ (model-null)/(1-null) ]: 0.002432501
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.07586      -0.05176
##
## Degrees of Freedom: 3362 Total (i.e. Null); 3361 Residual
## Null Deviance:      1404
## Residual Deviance: 1402 AIC: 1406
## log likelihood: -700.9344
## Nagelkerke R2: 0.001975013
## % pres/err predicted correctly: -339.4794
## % of predictable range [ (model-null)/(1-null) ]: 0.0007397835
## *****
## model index: 6
##

```

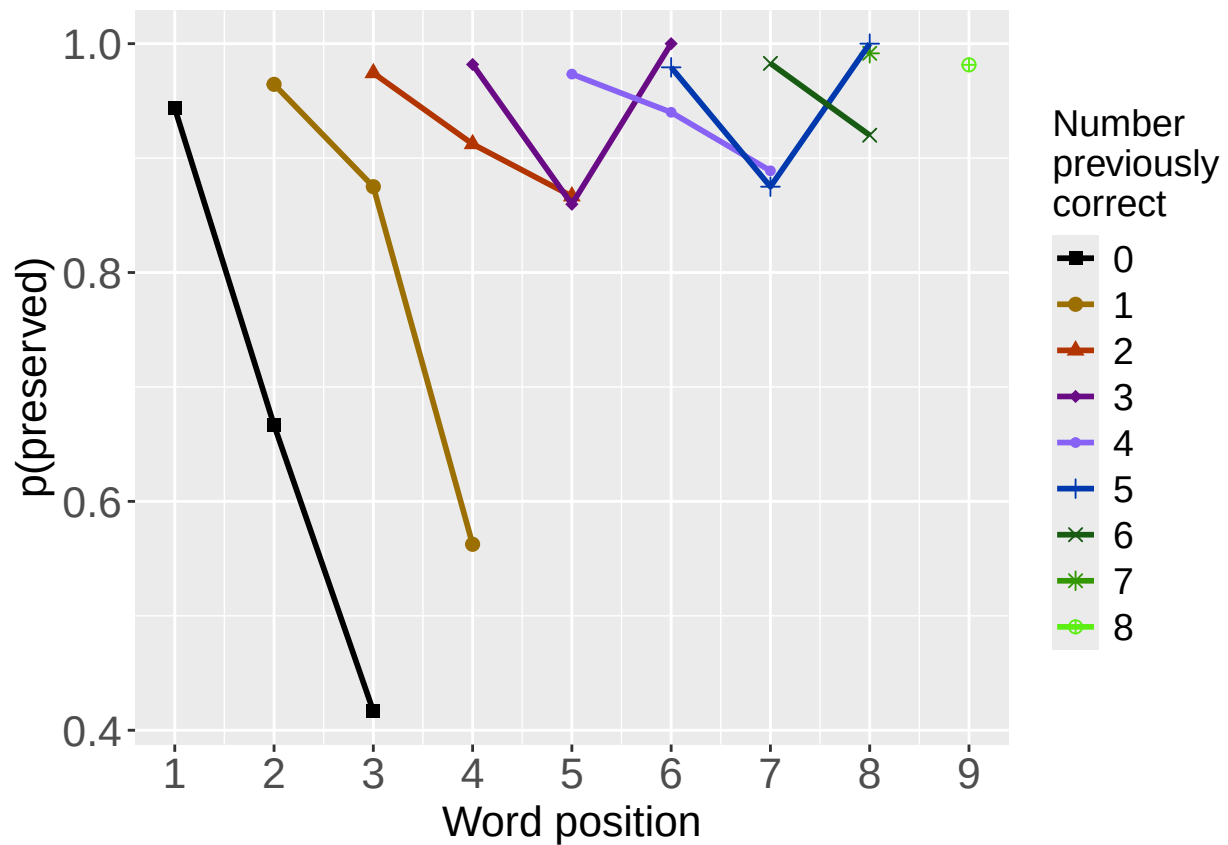
```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##       data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.873
##
## Degrees of Freedom: 3362 Total (i.e. Null);  3362 Residual
## Null Deviance:      1404
## Residual Deviance: 1404  AIC: 1406
## log likelihood:  -702.0683
## Nagelkerke R2:  -6.505372e-16
## % pres/err predicted correctly:  -339.7315
## % of predictable range [ (model-null)/(1-null) ]:  0
## *****
```

```
write.csv(SimpSyllMEAICSummary2,
          paste0(TablesDir, CurPat, "_", RunLabel, "_", CurTask,
                 "_OV_main_effects_model_summary.csv"), row.names = FALSE)
kable(SimpSyllMEAICSummary2)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumPres	CumErr	I(pos^2)	pos	stimlen
preserved ~ CumErr	1133.694	0.0000	1	1	0.2295918	16.406721	NA	-	NA	NA	NA
preserved ~ stimlen	1351.364	217.6702	0	0	0.0490435	33.908330	NA	NA	NA	NA	-
preserved ~ CumPres	1360.733	227.0383	0	0	0.0410072	2.294586	0.3529665	NA	NA	NA	NA
preserved ~ (I(pos^2) + pos)	1403.115	269.4209	0	0	0.0061102	2.636653	NA	NA	-	0.2254214	NA
preserved ~ pos	1405.869	272.1745	0	0	0.0019750	50.075862	NA	NA	NA	-	NA
preserved ~ 1	1406.137	272.4423	0	0	0.0000000	0.872623	NA	NA	NA	NA	NA

```
# plot prev err and prev cor plots
PrevCorPlot<-PlotObsPreviousCorrect(PosDat,palette_values,shape_values)
```

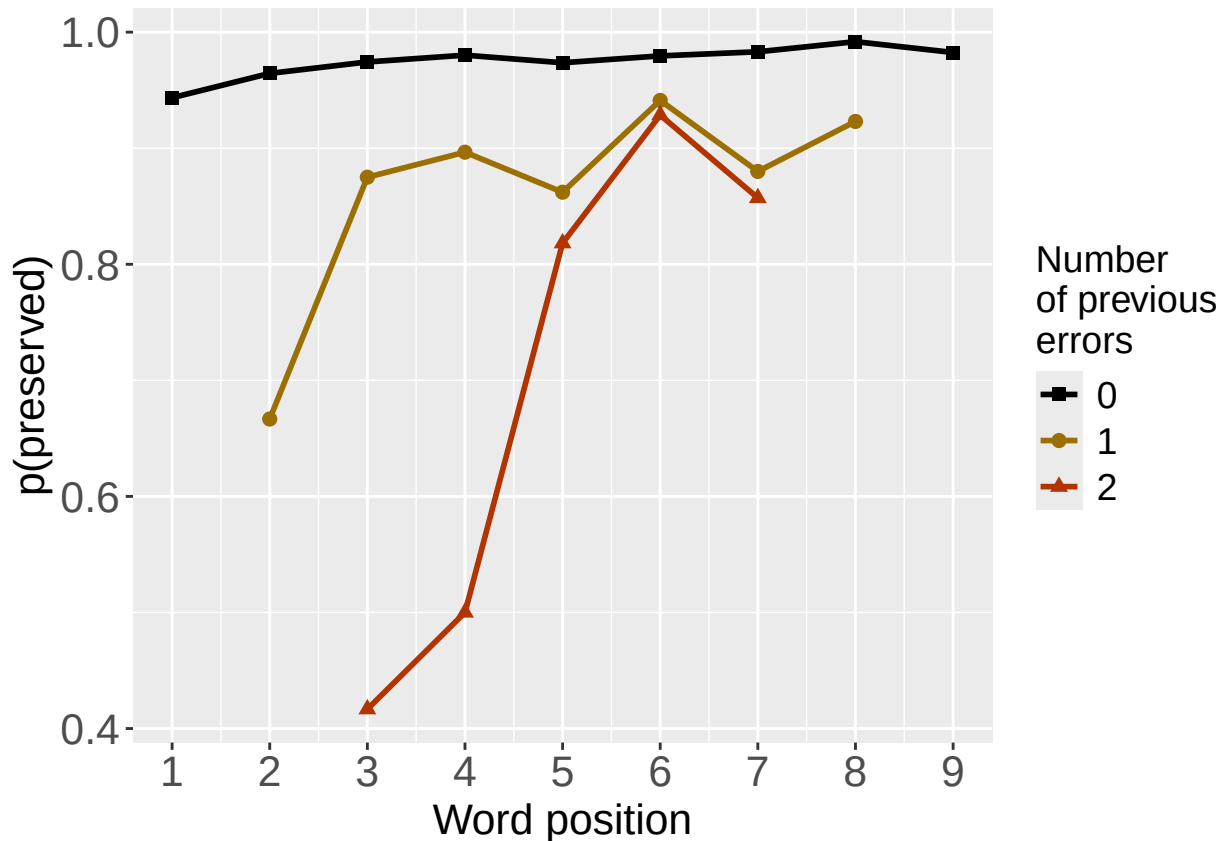
```
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPreviousError(PosDat,palette_values,shape_values)
```

```
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
```

```
print(PrevErrPlot)
```

```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevCorPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

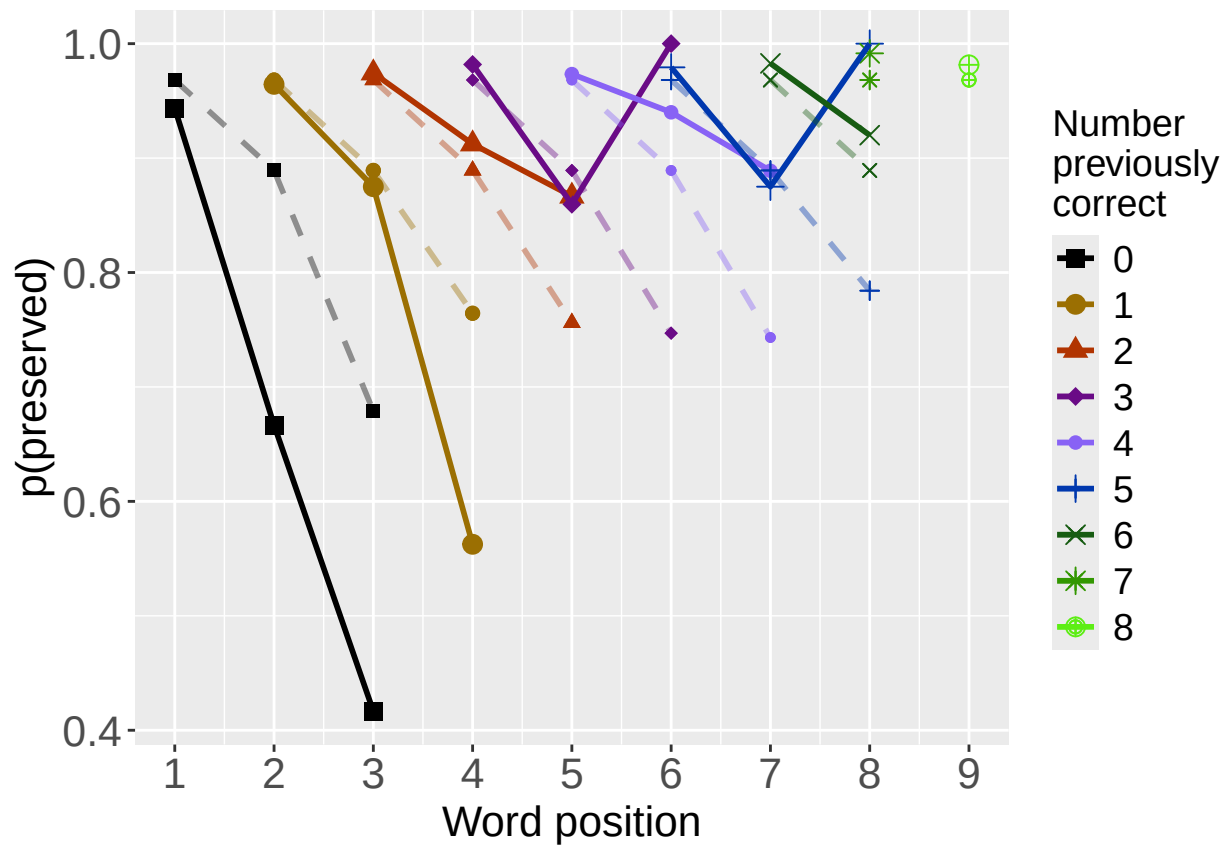
PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_PrevErrPlot_Obs")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image
# plot prev err and prev cor with predicted values
MEModel<-MERes$ModelResult[[ BestModelIndexL1 ]]
PosDat$MEPred<-fitted(MEModel)

PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","MEPred",palette_values,shape_values)

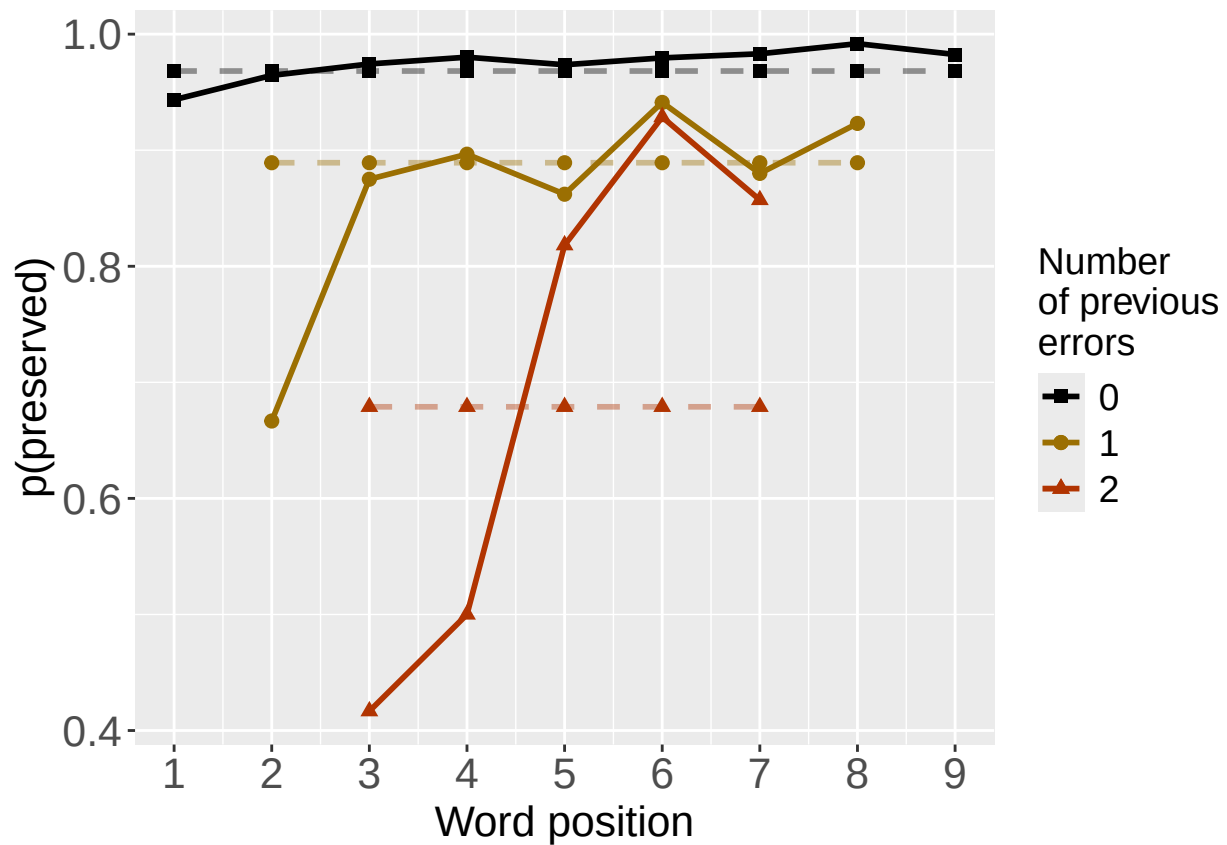
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)

```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","MEPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevCorPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevCorPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",
                  CurTask,"PrevErrPlot_ObsPredME")
ggsave(filename=paste0(PlotName,".tif"),plot=PrevErrPlot,device="tiff",compression = "lzw")

## Saving 6.5 x 4.5 in image

```

```

CumAICSummary <- NULL

#####
# level 2 -- Add position squared (quadratic with position)
#####

# After establishing the primary variable, see about additions

BestMEFactor<-BestMEModelFormula
OtherFactor<-"I(pos^2) + pos"
OtherFactorName<-"quadratic_by_pos"

if(!grepl("pos",BestMEFactor,fixed = TRUE)){ # if best model does not include pos
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor,OtherFactorName)
  CumAICSummary <- AICSummary
  kable(AICSummary)
}

```

```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      I(pos^2)         pos
##    2.40333    -1.56230    -0.02192     0.40712
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1386  AIC: 1394
## log likelihood:  -692.9438
## Nagelkerke R2:  0.2579844
## % pres/err predicted correctly:  -334.5104
## % of predictable range [ (model-null)/(1-null) ]:  0.2251782
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",

```

```

##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.417      -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1418  AIC: 1422
## log likelihood:  -709.0299
## Nagelkerke R2:  0.237682
## % pres/err predicted correctly:  -340.4165
## % of predictable range [ (model-null)/(1-null) ]:  0.2115386
## *****
## model index:  3
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      I(pos^2)      pos
##      2.6091      -0.0329      0.2332
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1767  AIC: 1773
## log likelihood:  -883.7294
## Nagelkerke R2:  0.006874509
## % pres/err predicted correctly:  -430.8122
## % of predictable range [ (model-null)/(1-null) ]:  0.002780514
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos
preserved ~ CumErr + I(pos^2) + pos	1393.888	0.00000	1e+00	0.9999992	0.2579844	2.403328	-1.562297	-0.0219209	0.4071182
preserved ~ CumErr	1422.060	28.17217	8e-07	0.0000008	0.2376820	3.416626	-1.333920	NA	NA
preserved ~ I(pos^2) + pos	1773.459	379.57117	0e+00	0.0000000	0.0068745	2.609117	NA	-0.0328963	0.2331777

```
#####
# level 2 -- Add length
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"stimlen"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr stimlen
## 4.9618 -1.2508 -0.1975
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4195 Residual
## Null Deviance: 1777
## Residual Deviance: 1403 AIC: 1409
## log likelihood: -701.4596
## Nagelkerke R2: 0.247256
## % pres/err predicted correctly: -339.1573
## % of predictable range [ (model-null)/(1-null) ]: 0.2144466
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept) CumErr
## 3.417 -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance: 1777
## Residual Deviance: 1418 AIC: 1422
## log likelihood: -709.0299
## Nagelkerke R2: 0.237682
## % pres/err predicted correctly: -340.4165
## % of predictable range [ (model-null)/(1-null) ]: 0.2115386
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
```

```
## (Intercept)      stimlen
##      5.8316      -0.3693
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1708  AIC: 1712
## log likelihood:  -854.1651
## Nagelkerke R2:  0.04729663
## % pres/err predicted correctly:  -425.1153
## % of predictable range [ (model-null)/(1-null) ]:  0.01593689
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	stimlen
preserved ~ CumErr + stimlen	1408.919	0.00000	1.0000000	0.9986006	0.2472560	4.961772	-	-
preserved ~ CumErr	1422.060	13.14067	0.0014013	0.0013994	0.2376820	3.416626	-	NA
preserved ~ stimlen	1712.330	303.41104	0.0000000	0.0000000	0.0472966	5.831551	NA	-
								0.3693428

```
#####
# level 2 -- add cumulative preserved
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"CumPres"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```

```
## *****
## model index:  2
##
## Call:  glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##      data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8885      -1.3179      0.2306
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1387  AIC: 1393
## log likelihood:  -693.4201
## Nagelkerke R2:  0.2573856
## % pres/err predicted correctly:  -334.0878
## % of predictable range [ (model-null)/(1-null) ]:  0.2261542
## *****
## model index:  1
##
```

```
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.417      -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1418 AIC: 1422
## log likelihood: -709.0299
## Nagelkerke R2:  0.237682
## % pres/err predicted correctly: -340.4165
## % of predictable range [ (model-null)/(1-null) ]:  0.2115386
## *****
## model index:  3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumPres
##      2.2255      0.2967
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1714 AIC: 1718
## log likelihood: -857.243
## Nagelkerke R2:  0.04311484
## % pres/err predicted correctly: -425.239
## % of predictable range [ (model-null)/(1-null) ]:  0.01565124
## *****
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres
preserved ~ CumErr + CumPres	1392.840	0.00000	1e+00	0.9999995	0.2573856	2.888516	- 1.317893	0.2305812
preserved ~ CumErr	1422.060	29.21967	5e-07	0.0000005	0.2376820	3.416626	- 1.333920	NA
preserved ~ CumPres	1718.486	325.64583	0e+00	0.0000000	0.0431148	2.225525	NA	0.2966624

```
#####
# level 2 -- Add linear position (NOT quadratic)
#####

BestMEFactor<-BestMEModelFormula
OtherFactor<-"pos"

if(!grepl(OtherFactor,BestMEFactor,fixed = TRUE)){
  AICSummary<-TestLevel2Models(PosDat,BestMEFactor,OtherFactor)
  CumAICSummary <- ConcatAndAddMissingColumns(CumAICSummary,AICSummary)
  kable(AICSummary)
}
```



```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      pos
##      2.6689      -1.5560      0.2266
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1388 AIC: 1394
## log likelihood: -693.7767
## Nagelkerke R2: 0.256937
## % pres/err predicted correctly: -334.3471
## % of predictable range [ (model-null)/(1-null) ]: 0.2255552
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##      3.417      -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1418 AIC: 1422
## log likelihood: -709.0299
## Nagelkerke R2: 0.237682
## % pres/err predicted correctly: -340.4165
## % of predictable range [ (model-null)/(1-null) ]: 0.2115386
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      pos
##      3.09117      -0.06055
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1774 AIC: 1778
## log likelihood: -886.8559
## Nagelkerke R2: 0.002566396
## % pres/err predicted correctly: -431.5924
## % of predictable range [ (model-null)/(1-null) ]: 0.0009788256
## *****

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	pos
preserved ~ CumErr	1393.553	0.00000	1e+00	0.9999994	0.2569370	2.668936	-	0.2266175
+ pos							1.556045	
preserved ~ CumErr	1422.060	28.50636	6e-07	0.0000006	0.2376820	3.416626	-	NA
							1.333920	
preserved ~ pos	1777.712	384.15835	0e+00	0.0000000	0.0025664	3.091172	NA	-
								0.0605511

```
CumAICSummary <- CumAICSummary %>% arrange(AIC)
BestModelFormulaL2 <- CumAICSummary$Model[BestModelIndexL2]
write.csv(CumAICSummary, paste0(TablesDir, CurPat, "_",
                                RunLabel, "_", CurTask,
                                "_main_effects_plus_one_model_summary.csv"), row.names = FALSE)
kable(CumAICSummary)
```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	I(pos^2)	pos	stimlen	CumPres
preserved ~ CumErr + CumPres	1392.840	0.00000	1.000000	0.9999999	0.2573855	2.688516	-	NA	NA	NA	0.2305812
							1.317893				
preserved ~ CumErr + pos	1393.553	0.00000	1.000000	0.9999999	0.2569370	2.668936	-	NA	0.2266175	NA	NA
							1.556045				
preserved ~ CumErr + I(pos^2) + pos	1393.888	0.00000	1.000000	0.9999999	0.2257984	2.4403328	-	-	0.4071182	NA	NA
							1.5622970	0.0219209			
preserved ~ CumErr + stimlen	1408.919	0.00000	1.000000	0.9986006	0.2472560	2.961772	-	NA	NA	-	NA
							1.250767			0.1975447	
preserved ~ CumErr	1422.060	8.172170	0.000000	0.0000000	0.2376820	3.416626	-	NA	NA	NA	NA
							1.333920				
preserved ~ CumErr	1422.060	3.140670	0.001401	0.0013990	0.2376820	3.416626	-	NA	NA	NA	NA
							1.333920				
preserved ~ CumErr	1422.060	9.219670	0.000000	0.0000000	0.2376820	3.416626	-	NA	NA	NA	NA
							1.333920				
preserved ~ CumErr	1422.060	8.506360	0.000000	0.0000000	0.2376820	3.416626	-	NA	NA	NA	NA
							1.333920				
preserved ~ stimlen	1712.338	03.411004	0.000000	0.0000000	0.0472966	5.831551	NA	NA	NA	-	NA
										0.3693428	
preserved ~ CumPres	1718.486	25.645830	0.000000	0.0000000	0.0431148	2.25525	NA	NA	NA	NA	0.2966624
preserved ~ I(pos^2) + pos	1773.459	79.571117	0.000000	0.0000000	0.0068745	6.09117	NA	-	0.2331777	NA	NA
								0.0328963			
preserved ~ pos	1777.712	384.15835	0.000000	0.0000000	0.0025664	3.091172	NA	NA	-	NA	NA
										0.0605511	

```
# explore influence of frequency and length
```

```
if(grepl("stimlen", BestModelFormulaL2)){ # if length is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2, " + log_freq")
  )
}
```

```

}else if(grepl("log_freq",BestModelFormulaL2)){ # if log_freq is already in the formula
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + stimlen")
  )
}else{
  Level3ModelEquations<-c(
    BestModelFormulaL2, # best model from level 2
    "preserved ~ 1", # null model
    paste0(BestModelFormulaL2," + log_freq"),
    paste0(BestModelFormulaL2," + stimlen"),
    paste0(BestModelFormulaL2," + stimlen + log_freq")
  )
}
}

Level3Res<-TestModels(Level3ModelEquations,PosDat)

```

```

## *****
## model index: 5
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      4.7894      -1.1485      0.2878      -0.2550      0.1499
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4193 Residual
## Null Deviance:      1777
## Residual Deviance: 1341 AIC: 1351
## log likelihood: -670.687
## Nagelkerke R2: 0.2858196
## % pres/err predicted correctly: -329.8437
## % of predictable range [ (model-null)/(1-null) ]: 0.2359554
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      5.0493      -1.1892      0.2850      -0.2936
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1354 AIC: 1362
## log likelihood: -677.0797
## Nagelkerke R2: 0.2778549
## % pres/err predicted correctly: -329.6714
## % of predictable range [ (model-null)/(1-null) ]: 0.2363532
## *****
## model index: 3

```

```

##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      log_freq
##      2.9063      -1.2368      0.2473      0.1944
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1364 AIC: 1372
## log likelihood: -682.1036
## Nagelkerke R2: 0.2715785
## % pres/err predicted correctly: -333.7044
## % of predictable range [ (model-null)/(1-null) ]: 0.2270394
## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8885      -1.3179      0.2306
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1387 AIC: 1393
## log likelihood: -693.4201
## Nagelkerke R2: 0.2573856
## % pres/err predicted correctly: -334.0878
## % of predictable range [ (model-null)/(1-null) ]: 0.2261542
## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)
##      2.853
##
## Degrees of Freedom: 4197 Total (i.e. Null);  4197 Residual
## Null Deviance:      1777
## Residual Deviance: 1777 AIC: 1779
## log likelihood: -888.7162
## Nagelkerke R2: 3.21633e-16
## % pres/err predicted correctly: -432.0162
## % of predictable range [ (model-null)/(1-null) ]: 0
## *****

```

```

BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[[BestModelIndexL3]]

```

```

AICSummary<-data.frame(Model=Level3Res$Model,
                        AIC=Level3Res$AIC,
                        row.names = Level3Res$Model)
AICSummary$DeltaAIC<-AICSummary$AIC-AICSummary$AIC[1]
AICSummary$AICexp<-exp(-0.5*AICSummary$DeltaAIC)
AICSummary$AICwt<-AICSummary$AICexp/sum(AICSummary$AICexp)
AICSummary$NagR2<-Level3Res$NagR2

AICSummary <- merge(AICSummary,Level3Res$CoefficientValues,
                    by='row.names',sort=FALSE)
AICSummary <- subset(AICSummary, select = -c(Row.names))

write.csv(AICSummary,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                           CurTask,"_main_effects_plus_one_len_freq_summary.csv"),
          row.names = FALSE)

kable(AICSummary)

```

Model	AIC	DeltaAIC	AICexp	AICwt	NagR2	(Intercept)	CumErr	CumPres	log_freq	stimlen
preserved ~ CumErr + CumPres + stimlen + log_freq	1351.374	0.00000	1.000000	0.995440	0.285819	6789371	-	0.287835	11499253	-
							1.148540			0.2550361
preserved ~ CumErr + CumPres + stimlen	1362.159	0.78523	0.004550	0.004529	0.277854	59049345	-	0.2849575	NA	-
							1.189223			0.2936398
preserved ~ CumErr + CumPres + log_freq	1372.202	0.83307	0.000029	0.000029	0.271578	5906272	-	0.247295	11944112	NA
							1.236808			
preserved ~ CumErr + CumPres	1392.840	1.46605	0.000000	0.000000	0.257385	56888516	-	0.2305812	NA	NA
							1.317893			
preserved ~ 1	1779.432	28.05827	0.000000	0.000000	0.000000	0.852547	NA	NA	NA	NA

```

# BestModelIndex is set by the parameter file and is used to choose
# a model that is essentially equivalent to the lowest AIC model, but
# simpler (and with a slightly higher AIC value, so not in first position)
BestModelL3<-Level3Res$ModelResult[[ BestModelIndexL3 ]]
BestModelFormulaL3<-Level3Res$Model[BestModelIndexL3]

```

```

BestModel<-BestModelL3
BestModelDeletions<-arrange(dropterm(BestModel),desc(AIC))
# order by terms that increase AIC the most when they are the one dropped
BestModelDeletions

```

```

## Single term deletions
##
## Model:
## preserved ~ CumErr + CumPres + stimlen + log_freq
##      Df Deviance   AIC
## CumErr    1   1577.9 1585.9
## CumPres    1   1390.7 1398.7
## stimlen    1   1364.2 1372.2
## log_freq    1   1354.2 1362.2
## <none>      1341.4 1351.4

```

```
#####
# Single deletions from best model
#####

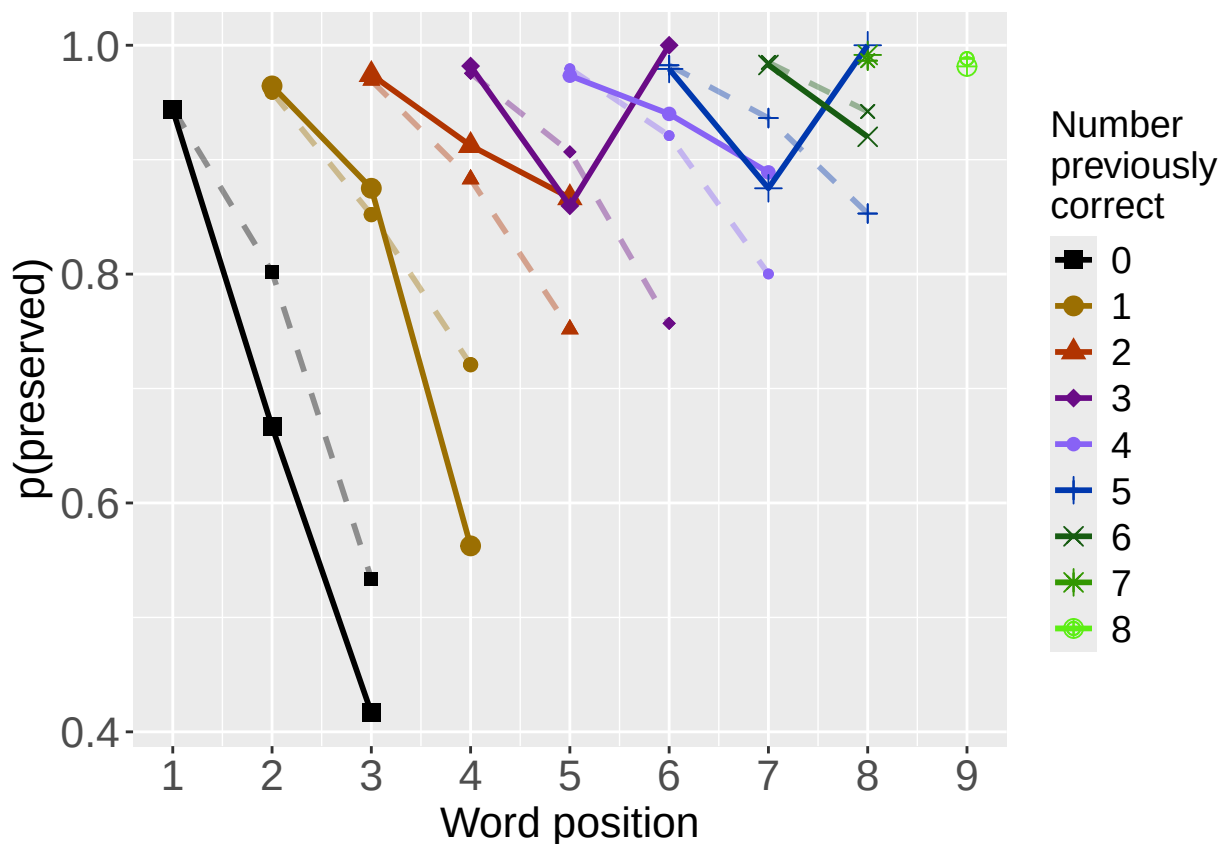
write.csv(BestModelDeletions,paste0(TablesDir,CurPat,"_",
                                   RunLabel,"_",CurTask,
                                   "_best_model_single_term_deletions.csv"),

          row.names = TRUE)

# plot prev err and prev cor with predicted values
PosDat$OAPred<-fitted(BestModel)

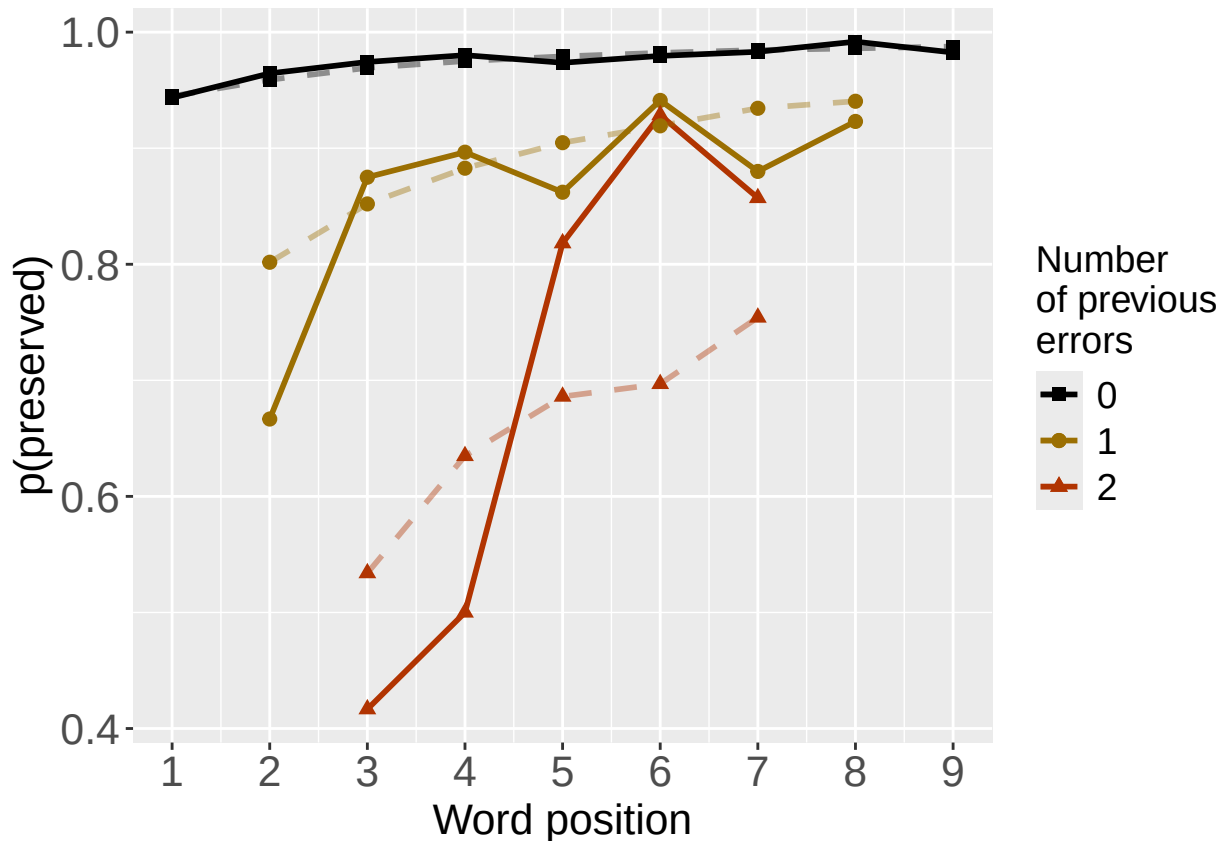
PrevCorPlot<-PlotObsPredPreviousCorrect(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
print(PrevCorPlot)
```



```
PrevErrPlot<-PlotObsPredPreviousError(PosDat,"preserved","OAPred",palette_values,shape_values)

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
print(PrevErrPlot)
```



```

PlotName<-paste0(FigDir,CurPat,"_",
                  RunLabel,"_",CurTask,
                  "_ObsPred_BestFullModel")
ggsave(filename=paste(PlotName,"_prev_correct.tif",sep=""),plot=PrevCorPlot,device="tiff",compression =

## Saving 6.5 x 4.5 in image
ggsave(filename=paste(PlotName,"_prev_error.tif",sep=""),plot=PrevErrPlot,device="tiff",compression = "

## Saving 6.5 x 4.5 in image
if(DoSimulations){
  BestModelFormulaL3Rnd <- BestModelFormulaL3
  if(grepl("CumPres",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumPres","RndCumPres",BestModelFormulaL3Rnd)
  }
  if(grepl("CumErr",BestModelFormulaL3Rnd)){
    BestModelFormulaL3Rnd <- gsub("CumErr","RndCumErr",BestModelFormulaL3Rnd)
  }

  RndModelAIC<-numeric(length=RandomSamples)
  for(rindex in seq(1,RandomSamples)){
    # Shuffle cumulative values
    PosDat$RndCumPres <- ShuffleWithinWord(PosDat,"CumPres")
    PosDat$RndCumErr <- ShuffleWithinWord(PosDat,"CumErr")
    BestModelRnd <- glm(as.formula(BestModelFormulaL3Rnd),
                        family="binomial",data=PosDat)
    RndModelAIC[rindex] <- BestModelRnd$aic
  }
}

```

```

}
ModelNames<-c(paste0("***",BestModelFormulaL3),
              rep(BestModelFormulaL3Rnd,RandomSamples))
AICValues <- c(BestModelL3$aic,RndModelAIC)
BestModelRndDF <- data.frame(Name=ModelNames,AIC=AICValues)
BestModelRndDF <- BestModelRndDF %>% arrange(AIC)
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random average"),
                                   AIC=c(mean(RndModelAIC))))
BestModelRndDF <- rbind(BestModelRndDF,
                        data.frame(Name=c("Random SD"),
                                   AIC=c(sd(RndModelAIC))))
write.csv(BestModelRndDF,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
                                "_best_model_with_random_cum_term.csv"),
          row.names = FALSE)
}

# plot influence factors
num_models <- nrow(BestModelDeletions) - 1
# minus 1 because last
# model is always <none> and we don't want to include that

if(num_models > 4){
  last_model_num <- 4
}else{
  last_model_num <- num_models
}

FinalModelSet<-ModelSetFromDeletions(BestModelFormulaL3,
                                     BestModelDeletions,
                                     last_model_num)

PlotName<-paste0(FigDir,CurPat,"_",RunLabel,"_",CurTask,"_FactorPlots")

FactorPlot<-PlotModelSetWithFreq(PosDat,FinalModelSet,
                                 palette_values,FinalModelSet,PptID=CurPat)

## *****
## model index: 1
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
##          data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr
##          3.417      -1.334
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4196 Residual
## Null Deviance:      1777
## Residual Deviance: 1418 AIC: 1422
## log likelihood: -709.0299
## Nagelkerke R2: 0.237682
## % pres/err predicted correctly: -340.4165
## % of predictable range [ (model-null)/(1-null) ]: 0.2115386

```



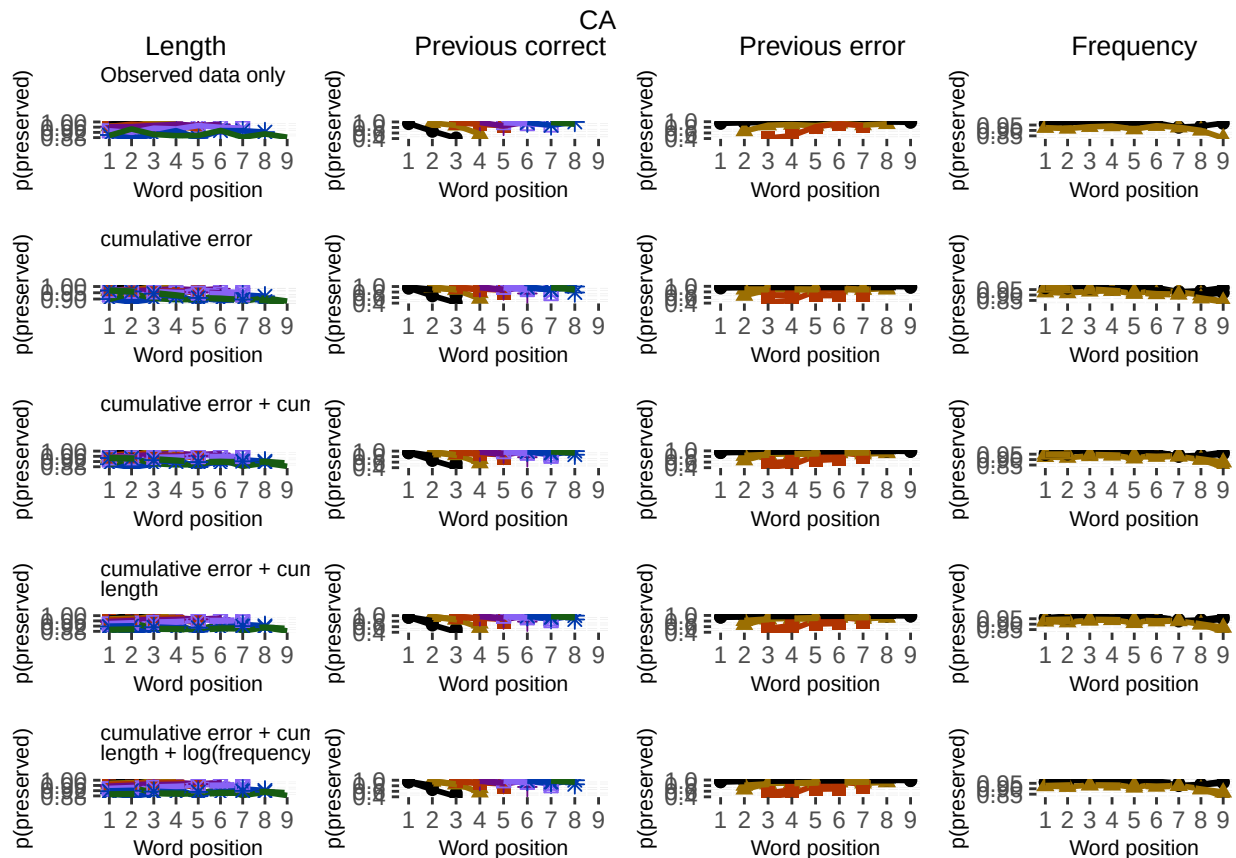
```

## *****
## model index: 2
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres
##      2.8885      -1.3179      0.2306
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4195 Residual
## Null Deviance:      1777
## Residual Deviance: 1387 AIC: 1393
## log likelihood: -693.4201
## Nagelkerke R2: 0.2573856
## % pres/err predicted correctly: -334.0878
## % of predictable range [ (model-null)/(1-null) ]: 0.2261542
## *****
## model index: 3
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen
##      5.0493      -1.1892      0.2850      -0.2936
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4194 Residual
## Null Deviance:      1777
## Residual Deviance: 1354 AIC: 1362
## log likelihood: -677.0797
## Nagelkerke R2: 0.2778549
## % pres/err predicted correctly: -329.6714
## % of predictable range [ (model-null)/(1-null) ]: 0.2363532
## *****
## model index: 4
##
## Call: glm(formula = as.formula(ModelEquations[Index]), family = "binomial",
## data = PosDat)
##
## Coefficients:
## (Intercept)      CumErr      CumPres      stimlen      log_freq
##      4.7894      -1.1485      0.2878      -0.2550      0.1499
##
## Degrees of Freedom: 4197 Total (i.e. Null); 4193 Residual
## Null Deviance:      1777
## Residual Deviance: 1341 AIC: 1351
## log likelihood: -670.687
## Nagelkerke R2: 0.2858196
## % pres/err predicted correctly: -329.8437
## % of predictable range [ (model-null)/(1-null) ]: 0.2359554
## *****
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```



```
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 7 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 9 rows containing missing values or values outside the scale range (`geom_point()`)
## Warning: The shape palette can deal with a maximum of 6 discrete values because more than 6 becomes
## i you have requested 9 values. Consider specifying shapes manually if you need that many have them.
## Warning: Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
## Removed 4 rows containing missing values or values outside the scale range (`geom_point()`)
ggsave(paste0(PlotName, ".tif"), plot=FactorPlot, width = 360, height=400, units="mm", device="tiff", compress=
# use \blandscape and \elandscape to make markdown plots landscape if needed
FactorPlot
```



```
DA.Result<-dominanceAnalysis(BestModel)
DAContributionAverage<-ConvertDominanceResultToTable(DA.Result)
write.csv(DAContributionAverage,paste0(TablesDir,CurPat,"_",RunLabel,"_",
CurTask,"_dominance_analysis_table.csv"),
row.names = TRUE)
kable(DAContributionAverage)
```

	CumErr	CumPres	stimlen	log_freq
McFadden	0.1676687	0.0343107	0.0265808	0.0167703

	CumErr	CumPres	stimlen	log_freq
SquaredCorrelation	0.0670879	0.0138608	0.0108533	0.0068581
Nagelkerke	0.0670879	0.0138608	0.0108533	0.0068581
Estrella	0.0773017	0.0156373	0.0119083	0.0075017

	deviance	deviance_explained
CumErr + CumPres + stimlen + log_freq	1341.374	436.0583
CumErr + CumPres + stimlen	1354.159	423.2730
CumErr + CumPres	1386.840	390.5922
CumErr	1418.060	359.3726
null	1777.432	0.0000

```
deviance_differences_df<-GetDevianceSet(FinalModelSet,PosDat)
write.csv(deviance_differences_df,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                                         CurTask,"_deviance_differences_table.csv"),
          row.names = FALSE)
deviance_differences_df
```

```
##                                     model deviance deviance_explained percent_explained
## CumErr + CumPres + stimlen + log_freq CumErr + CumPres + stimlen + log_freq 1341.374      436.0583      24.53304
## CumErr + CumPres + stimlen              CumErr + CumPres + stimlen 1354.159      423.2730      23.81374
## CumErr + CumPres                        CumErr + CumPres 1386.840      390.5922      21.97508
## CumErr                                CumErr 1418.060      359.3726      20.21863
## null                                  null 1777.432      0.0000      0.00000
##                                     percent_of_explained_deviance increment_in_explained
## CumErr + CumPres + stimlen + log_freq      100.00000      2.931999
## CumErr + CumPres + stimlen                  97.06800      7.494600
## CumErr + CumPres                          89.57340      7.159518
## CumErr                                    82.41388      82.413882
## null                                      NA      0.000000
```

```
kable(deviance_differences_df[,2:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
kable(deviance_differences_df[,c(4:6)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
NagContributions <- t(DAContributionAverage[3,])
NagTotal<-sum(NagContributions)
NagPercents <- NagContributions / NagTotal
NagResultTable <- data.frame(NagContributions = NagContributions, NagPercents = NagPercents)
names(NagResultTable)<-c("NagContributions","NagPercents")
write.csv(NagResultTable,paste0(TablesDir,CurPat,"_",RunLabel,"_",CurTask,
```

	percent_explained	percent_of_explained_deviance	increment_in_explained
CumErr + CumPres + stimlen + log_freq	24.53304	100.00000	2.931999
CumErr + CumPres + stimlen	23.81374	97.06800	7.494600
CumErr + CumPres	21.97508	89.57340	7.159518
CumErr	20.21863	82.41388	82.413882
null	0.00000	NA	0.000000

```

                                "_nagelkerke_contributions_table.csv"),row.names = TRUE)
NagPercents

```

```

##           Nagelkerke
## CumErr    0.67999025
## CumPres   0.14049007
## stimlen   0.11000730
## log_freq  0.06951237

```

```

sse_results_list<-compare_SS_accounted_for(FinalModelSet,"preserved ~ 1",PosDat,N_cutoff=10)

```

```

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (null_cumcor_resid^2) * (N_values$len_pos_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.

```

```

## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'stimlen'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumErr'. You can override using the `.groups` argument.
## `summarise()` has grouped output by 'CumPres'. You can override using the `.groups` argument.

## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length
## Warning in (cumerr_residuals^2) * (N_values$cumerr_N): longer object length is not a multiple of shorter object length

sse_table<-sse_results_table(sse_results_list)
write.csv(sse_table,paste0(TablesDir,CurPat,"_",RunLabel,"_",
                          CurTask,"_sse_results_table.csv"),row.names = TRUE)

sse_table

##               model p_accounted_for model_deviance diff_CumErr diff_CumErr+CumPres
## 1      preserved ~ CumErr      0.7690523      1418.060  0.00000000      -0.09947423
## 2      preserved ~ CumErr+CumPres      0.8685265      1386.840  0.09947423      0.00000000
## 3 preserved ~ CumErr+CumPres+stimlen+log_freq      0.8983048      1341.374  0.12925253      0.02977830
## 4      preserved ~ CumErr+CumPres+stimlen      0.9034658      1354.159  0.13441356      0.03493933
## diff_CumErr+CumPres+stimlen+log_freq diff_CumErr+CumPres+stimlen
## 1      -0.129252526      -0.134413556
## 2      -0.029778297      -0.034939328
## 3      0.000000000      -0.005161031
## 4      0.005161031      0.000000000

```

model	p_accounted_for	model_deviance
preserved ~ CumErr	0.7690523	1418.060
preserved ~ CumErr+CumPres	0.8685265	1386.840
preserved ~ CumErr+CumPres+stimlen+log_freq	0.8983048	1341.374
preserved ~ CumErr+CumPres+stimlen	0.9034658	1354.159

model	diff_CumErr	diff_CumErr+CumPres	diff_CumErr+CumPres+stimlen+log_freq	diff_CumErr+CumPres+stimlen
preserved ~ CumErr	0.0000000	-0.0994742	-0.1292525	-0.1344136
preserved ~ CumErr+CumPres	0.0994742	0.0000000	-0.0297783	-0.0349393
preserved ~ CumErr+CumPres+stimlen+log_freq	0.1292525	0.0297783	0.0000000	-0.0051610
preserved ~ CumErr+CumPres+stimlen	0.1344136	0.0349393	0.0051610	0.0000000

```
kable(sse_table[,1:3], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```

```
write.csv(results_report_DF, paste0(TablesDir, CurPat, "_", RunLabel, "_",
  CurTask, "_results_report_df.csv"), row.names = FALSE)
```

```
ncols_in_table <- ncol(sse_table)
kable(sse_table[,c(1,4:ncols_in_table)], format="latex", booktabs=TRUE) %>%
  kable_styling(latex_options="scale_down")
```